TÀI LIỆU ÔN TẬP MÔN HỌC

CƠ SỞ VÀ THIẾT KẾ HỆ THỐNG IoT

TỔNG HỢP KIẾN THỨC CHI TIẾT

Sáu chương độc lập từ hai bài giảng; sơ đồ TikZ tự biên soạn

Nguồn chính: “Chapter 1: An overview of IoT” và “Chapter 2: IoT Hardware”
Học phần EEE703006, TS. Nguyễn Đắc Cử, Khoa Điện – Điện tử, Trường Đại học Phenikaa.

Có đối chiếu tài liệu của Hassan (2018); McEwen–Cassimally (2014); James–Seth–Mukhopadhyay (2022).

2026

Lời nói đầu và phạm vi biên soạn

Tài liệu này **không giả định đã có bản giảng chương 2 và chương 3 riêng**. Hai tệp bài giảng được **tái tổ chức thành sáu chương học tập**. *Chapter 1: An overview of IoT* (50 trang) tạo thành chương 1–3 về khái niệm, thành phần kiến trúc và ứng dụng. *Chapter 2: IoT Hardware* (79 trang) tạo thành chương 4–6 về cảm biến, cơ cấu chấp hành, vi điều khiển và các nền tảng IoT. Cách phân chia chỉ nhằm học và quản lý phiên bản, không phải đánh số chương gốc của giảng viên.

Các phần **"Kiến thức mở rộng"** được thêm để giải thích kỹ thuật, ví dụ minh họa và mối liên hệ giữa các khái niệm; không được hiểu là trích nguyên văn bài giảng. Các sơ đồ là hình **tự biên soạn bằng TikZ** từ nội dung kiến thức, không sao chép hình bản quyền của sách. Những dự báo lịch sử trong slide không được trình bày như số liệu thực tế năm 2026.

Ghi nhớ

Mục tiêu sử dụng: đọc lần lượt sáu chương, tự kiểm tra bằng câu hỏi cuối chương, sau đó dùng bảng đối chiếu thuật ngữ và phụ lục nguồn để ôn tập. Khi có tài liệu ba chương tiếp theo, thêm tệp vào thư mục `chapters/` và khai báo `\input` trong `main.tex`; không phải sửa sáu chương hiện tại.

Mục lục

Lời nói đầu và phạm vi biên soạn **i**

1 Tổng quan, lịch sử và khái niệm nền tảng của IoT **1**
1.1 Tổng quan về học phần và phạm vi IoT 1
1.1.1 IoT là gì? 1
1.2 Lịch sử hình thành Internet và IoT 2
1.2.1 Bốn giai đoạn phát triển Internet theo bài giảng 2
1.2.2 Các mốc đáng ghi nhớ 3
1.3 Đối tượng thông minh và môi trường thông minh 3
1.3.1 Smart Object (SO) – đối tượng thông minh 3
1.3.2 Chức năng cốt lõi và xu hướng phát triển 4
1.3.3 Smart Environment (SE) – môi trường thông minh 4
1.4 Những khái niệm liên quan: M2M, IIoT, Industry 4.0 4
1.4.1 Machine-to-Machine (M2M) 4
1.4.2 Industrial Internet of Things (IIoT) 4
1.4.3 Industry 4.0 – công nghiệp 4.0 4
1.5 Tư duy hệ thống: từ dữ liệu tới hành động 5
1.6 Tổng kết và câu hỏi tự kiểm tra 5

2 Thành phần hệ thống và các mô hình kiến trúc IoT **6**
2.1 Tư duy thiết kế theo kiến trúc tổng thể 6
2.2 Bốn khối chức năng và giao diện người dùng 6
2.2.1 Things/Devices – thiết bị đầu cuối 7
2.2.2 Gateway/Network – cổng kết nối và liên mạng 7
2.2.3 Cloud – hạ tầng lưu trữ và tính toán 7
2.2.4 Services – dịch vụ và phân tích nghiệp vụ 7
2.2.5 User Interface (UI) – giao diện người dùng 7
2.3 Kiến trúc 3 lớp (three-layer architecture) 8
2.3.1 Perception layer – lớp cảm nhận 8
2.3.2 Network layer – lớp mạng 8
2.3.3 Application layer – lớp ứng dụng 8
2.4 Kiến trúc 4 lớp hướng dịch vụ (SoA) 9
2.5 Kiến trúc 5 lớp (five-layer architecture) 9
2.6 Kiến trúc 6 và 7 lớp trong hình của bài giảng 10

2.6.1 Sáu lớp: chú ý cách đếm trong chính slide . . . 10
2.6.2 Bảy lớp: dữ liệu và giá trị được tách chi tiết hơn . . . 11
2.7 Bảng đối chiếu nhanh các mô hình . . . 12
2.8 Ví dụ đặt các thành phần vào từng lớp . . . 12
2.9 Các chỉ tiêu thiết kế cần đánh giá . . . 13
2.9.1 Thông lượng và tần suất gửi . . . 13
2.9.2 Độ trễ, độ tin cậy và tính khả dụng . . . 13
2.9.3 Năng lượng của nút IoT . . . 13
2.10 Tổng kết và câu hỏi tự kiểm tra . . . 13

3 Ứng dụng, cơ hội và thách thức của Internet of Things 15
3.1 Các lĩnh vực ứng dụng IoT . . . 15
3.1.1 Nhà thông minh và tòa nhà thông minh . . . 15
3.1.2 Thiết bị đeo và y tế kết nối . . . 16
3.1.3 Đô thị thông minh và giao thông . . . 16
3.1.4 Công nghiệp, điện và chuỗi cung ứng . . . 16
3.1.5 Bán lẻ và nông nghiệp thông minh . . . 16
3.2 Tình huống 1 trong bài giảng: đồng hồ báo thức thông minh . . . 16
3.3 Tình huống 2 trong bài giảng: ô tô kết nối . . . 17
3.4 Cơ hội do IoT mang lại . . . 17
3.4.1 Thị trường mới (New Markets) . . . 17
3.4.2 Thay đổi cách tạo giá trị (Changed Value Creation) . . . 18
3.4.3 Nhận biết rõ hơn không gian thông tin . . . 18
3.5 Các thách thức công nghệ trong triển khai IoT . . . 18
3.5.1 Kết nối mọi nơi – mạng không luôn sẵn sàng . . . 18
3.5.2 Chi phí thấp và giới hạn tài nguyên phần cứng . . . 18
3.5.3 Năng lượng và power management . . . 18
3.5.4 Scalability – khả năng mở rộng . . . 19
3.5.5 Reliability và fault tolerance . . . 19
3.5.6 Security – an ninh hệ thống . . . 19
3.6 Thách thức xã hội, đạo đức, pháp lý và rủi ro . . . 19
3.6.1 Sự chấp nhận của cá nhân và cộng đồng . . . 19
3.6.2 Dữ liệu và quyền riêng tư . . . 19
3.6.3 Rủi ro trong vòng đời sản phẩm . . . 20
3.7 Đối chiếu cơ hội – yêu cầu thiết kế . . . 20
3.8 Phân tích dự báo trong slide và giới hạn sử dụng . . . 20
3.9 Tổng kết: từ IoT tới giá trị thực tế . . . 20
3.10 Câu hỏi tự kiểm tra và bài tập tình huống . . . 21

4 Cảm biến (Sensors) 22
4.1 Mục tiêu học tập và vị trí trong kiến trúc IoT . . . 22
4.2 Khái niệm và đặc tính chuyển đổi . . . 22
4.2.1 Các đại lượng ảnh hưởng chất lượng phép đo (mở rộng kỹ thuật) . . . 23
4.3 Phân loại cảm biến . . . 23

4.3.1 Cảm biến tích cực và thụ động . . . 23
4.3.2 Tín hiệu tương tự và tín hiệu số . . . 24
4.3.3 Phân loại theo đại lượng và cơ chế . . . 24
4.4 Kết nối cảm biến với vi điều khiển (mở rộng kỹ thuật) . . . 24
4.4.1 ADC, lượng tử hóa và ví dụ tính toán . . . 25
4.4.2 Cảm biến điện trở: mạch phân áp . . . 25
4.4.3 I^2C, SPI, UART và ngõ vào số . . . 25
4.5 Mô hình nút cảm biến IoT . . . 26
4.5.1 Nguồn và độ tin cậy đo . . . 26
4.6 Ví dụ thiết kế: đo nhiệt độ và điều khiển quạt . . . 26
4.7 Câu hỏi ôn tập và bài tập . . . 27

5 Cơ cấu chấp hành (Actuators) 28
5.1 Mục tiêu và khái niệm . . . 28
5.2 So sánh cảm biến và cơ cấu chấp hành . . . 28
5.3 Nhóm phần tử chấp hành thường gặp . . . 28
5.3.1 LED, màn hình và còi báo . . . 28
5.3.2 Động cơ DC, động cơ bước và servo . . . 29
5.3.3 Rơ-le, van điện từ, tải nhiệt và cơ cấu tuyến tính . . . 29
5.4 Nguyên lý ghép mạch điều khiển công suất . . . 29
5.4.1 Vì sao GPIO không cấp trực tiếp được tải lớn? . . . 29
5.4.2 PWM: điều chỉnh công suất trung bình . . . 30
5.5 Đặc tính kỹ thuật khi chọn actuator . . . 30
5.6 Điều khiển vòng hở và vòng kín . . . 31
5.7 Ví dụ chương trình Arduino: LED bằng PWM . . . 31
5.8 Bài tập và câu hỏi ôn tập . . . 31

6 Vi điều khiển và nền tảng phần cứng IoT 33
6.1 Kiến trúc hệ thống nhúng và mục tiêu học tập . . . 33
6.2 ATmega328P: nền tảng AVR 8 bit . . . 34
6.2.1 Flash, SRAM, EEPROM: ba vai trò khác nhau . . . 34
6.2.2 Chân nguồn, I/O và nạp chương trình . . . 34
6.3 Arduino: hệ sinh thái phần cứng/phần mềm mở . . . 34
6.3.1 Các bo Arduino trong bài giảng . . . 34
6.3.2 Phân tích một bo Uno cổ điển . . . 35
6.3.3 Shield, breadboard, kit và các nền tảng thay thế . . . 35
6.4 Lập trình Arduino IDE và ngôn ngữ Sketch . . . 36
6.4.1 Chu trình công cụ và chương trình . . . 36
6.4.2 Hai hàm `setup()` và `loop()` . . . 36
6.4.3 Lệnh và khái niệm quan trọng . . . 36
6.4.4 Ví dụ: đọc analog và gửi Serial . . . 37
6.4.5 Serial UART và COM . . . 37
6.5 ESP8266 và các module phổ biến . . . 37
6.5.1 ESP-01, ESP-07 và NodeMCU . . . 38

6.5.2 Firmware, SDK và quy trình IDE 38
6.6 ESP32: một họ SoC, không phải một chip duy nhất 38
6.6.1 Lợi thế và giới hạn thực dụng 38
6.6.2 So sánh các lựa chọn MCU căn bản 39
6.7 Intel Edison: ví dụ nền tảng lai có tính lịch sử 39
6.8 Edge Computing trong kiến trúc IoT 39
6.8.1 So sánh các vị trí xử lý 40
6.9 Raspberry Pi: máy tính đơn bo cho gateway và Edge 40
6.9.1 Các thành phần và GPIO 40
6.9.2 Dựng gateway cơ bản với Raspberry Pi 41
6.9.3 Arduino và Raspberry Pi: không có nền tảng nào tốt nhất cho mọi việc . 41
6.10 Thiết kế nút IoT: bài toán tổng hợp nhiều nền tảng 41
6.10.1 Bảng quyết định chọn phần cứng 41
6.10.2 Công thức sơ bộ về lưu trữ và truyền thông 42
6.10.3 Ví dụ hệ IoT đầy đủ: giám sát phòng học 42
6.11 Câu hỏi tự luyện và đáp án ngắn 42
6.12 Bảng thuật ngữ ôn thi 43

Tài liệu tham khảo và quy ước trích dẫn **45**

CHƯƠNG 1

Tổng quan, lịch sử và khái niệm nền tảng của IoT

Ghi nhớ

Mục tiêu: (i) trình bày khái niệm Internet of Things (IoT); (ii) giải thích vì sao IoT xuất hiện; (iii) phân biệt smart object, smart environment, ubiquitous computing, M2M, IIoT và Industry 4.0; (iv) mô tả chu trình cảm nhận–xử lý–kết nối–tác động.

1.1 Tổng quan về học phần và phạm vi IoT

Học phần *Cơ sở và thiết kế hệ thống IoT* (mã EEE703006) giới thiệu nền tảng phần cứng/phần mềm, các giao thức truyền thông và cơ chế xử lý dữ liệu. Bài giảng dẫn các ví dụ Zigbee, Bluetooth cùng các họ chuẩn IEEE 802.15.4, IEEE 802.15.6 và WLAN. Phần lịch học theo khóa 2022 trong bài giảng là thông tin hành chính, không phải nội dung kiến thức cần học thuộc.

Theo bài giảng Chapter 1: An overview of IoT, trang PDF 2–4.

1.1.1 IoT là gì?

Internet of Things (IoT) là hệ thống gồm những đối tượng vật lý hoặc thiết bị có khả năng nhận dạng, thu thập thông tin, trao đổi dữ liệu qua mạng và, tùy ứng dụng, tự động thực hiện hành động dựa trên dữ liệu. Trong bài giảng, đối tượng có thể là thiết bị, máy móc, động vật được gắn thẻ, phương tiện hoặc con người gắn thiết bị theo dõi. Các đối tượng không nhất thiết cần thao tác nhập liệu trực tiếp từ con người trong từng lần trao đổi.

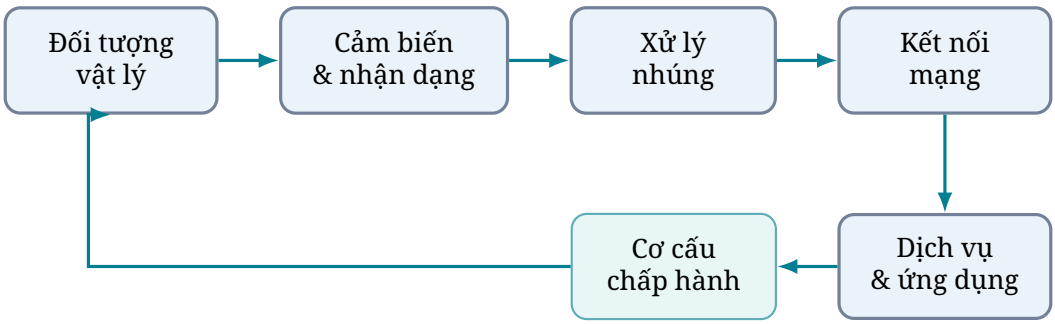
Cần phân biệt ba cấp độ:

1. **Thiết bị điện tử tự động:** cảm biến và bộ điều khiển hoạt động cục bộ, không có trao đổi thông tin qua mạng; chưa đủ để khẳng định là một hệ thống IoT.
2. **Thiết bị được kết nối:** có khả năng gửi/nhận dữ liệu bằng một kênh truyền, kể cả qua thiết bị trung gian (gateway).
3. **Hệ thống IoT:** liên kết thiết bị và dữ liệu với dịch vụ, quản lý, ứng dụng và quy trình phản hồi tạo ra giá trị sử dụng.

Ví dụ: cảm biến nhiệt độ điều khiển quạt tại chỗ là hệ thống nhúng tự động; nếu bổ sung khả năng gửi số đo tới máy chủ, theo dõi từ xa và cấu hình ngưỡng qua ứng dụng, ta đã có chức năng đặc trưng của IoT. **Không phải mọi “thing” phải sở hữu địa chỉ IP công khai**: nhiều nút chỉ liên lạc cục bộ, gateway chịu trách nhiệm kết nối Internet. Đây là cách hiểu kỹ

thuật chính xác hơn so với diễn đạt rút gọn “mọi thiết bị có nút ON/OFF đều có thể kết nối Internet” trong slide.

Theo bài giảng Chapter 1: An overview of IoT, trang PDF 5, 12–14.



Hình 1.1: Chu trình điển hình của IoT; đường phản hồi tới môi trường là tùy chọn. Sơ đồ tự biên soạn.

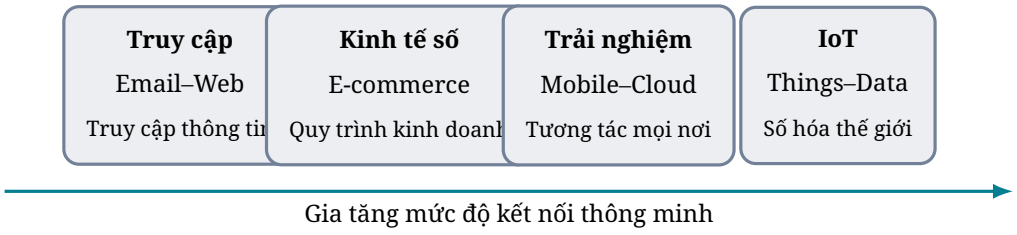
1.2 Lịch sử hình thành Internet và IoT

1.2.1 Bốn giai đoạn phát triển Internet theo bài giảng

Bài giảng dùng một cách phân kỳ theo **mức độ tác động và mục đích sử dụng**, không phải các giai đoạn có ranh giới thời gian tuyệt đối. Đây là tiến trình từ số hóa khả năng truy cập thông tin tới số hóa thế giới vật lý.

Giai đoạn	Trọng tâm	Ví dụ và hệ quả
Connectivity – Digitize access	Kết nối con người tới thông tin	Email, trình duyệt web, tìm kiếm; việc truy cập tài nguyên số trở nên dễ dàng.
Networked economy – Digitize business	Số hóa hoạt động kinh doanh	Thương mại điện tử, chuỗi cung ứng, phối hợp nghiệp vụ, tối ưu hiệu suất.
Immersive experiences – Digitize interactions	Số hóa tương tác, trải nghiệm	Di động, mạng xã hội, video, đám mây; truy cập dịch vụ liên tục.
Internet of Things – Digitize the world	Kết nối đối tượng vật lý	Dữ liệu từ cảm biến và máy móc trở thành nguồn dịch vụ; liên thông giữa hệ vật lý và thế giới số.

Theo bài giảng Chapter 1: An overview of IoT, trang PDF 6–8.



Hình 1.2: Bốn giai đoạn nhìn theo mục tiêu ứng dụng Internet.

1.2.2 Các mốc đáng ghi nhớ

- **Thập niên 1970–1980:** các đối tượng đặc thù như máy ATM và máy bán hàng có kết nối là tiền thân của tư duy thiết bị được mạng hóa. Bài giảng nêu ATM từ 1974 và máy bán hàng kết nối tại Carnegie Mellon vào thập niên 1980.
- **1999:** Kevin Ashton được ghi nhận đã sử dụng thuật ngữ *Internet of Things* trong bối cảnh quản lý chuỗi cung ứng tại Procter & Gamble, gắn với RFID để nhận diện và theo dõi sản phẩm.
- **1999:** Neil Gershenfeld xuất bản *When Things Start to Think*; bài giảng dùng tác phẩm này để minh họa tầm nhìn về các vật thể có năng lực tính toán.
- **2008–2009:** được slide coi là thời điểm IoT bắt đầu được nhìn nhận như một kỷ nguyên mới; đây là nhận định về độ phổ biến của khái niệm, *không phải* ngày phát minh của IoT.
- **Khoảng từ 2010:** Industrial Internet of Things (IIoT) được thảo luận rộng hơn trong ứng dụng công nghiệp.

Theo bài giảng Chapter 1: An overview of IoT, trang PDF 9–11.

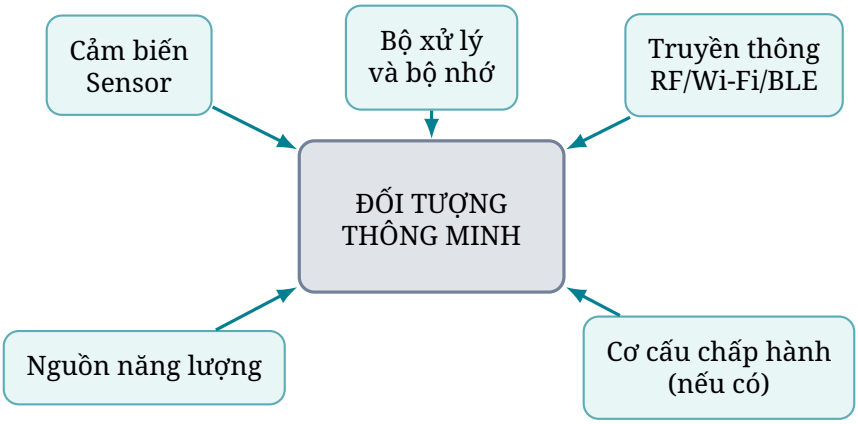
Ghi nhớ

Câu hỏi hay gặp: Kevin Ashton không "phát minh ra Internet" năm 1999. Ông thường được ghi nhận là người đặt tên *Internet of Things*. Sự phát triển IoT dựa trên nhiều công nghệ đã có trước đó như cảm biến, RFID, mạng máy tính, vi điều khiển và truyền thông không dây.

1.3 Đối tượng thông minh và môi trường thông minh

1.3.1 Smart Object (SO) – đối tượng thông minh

Một **smart object** là vật thể có tích hợp một số thành phần: bộ xử lý; bộ nhớ hoặc phương tiện lưu dữ liệu; cảm biến; giao tiếp mạng; nguồn cấp điện. Tùy chức năng, nó còn có **actuator** để tác động lên môi trường. Thiết bị có thể nhận dữ liệu, xử lý theo logic đã lập trình, truyền thông và thay đổi trạng thái vật lý. Ví dụ: đồng hồ đo điện thông minh, máy cảnh báo khói, cảm biến chất lượng không khí, công tắc chiếu sáng kết nối.



Hình 1.3: Các khối của một smart object (kiến trúc chức năng).

1.3.2 Chức năng cốt lõi và xu hướng phát triển

Bài giảng đề cập khả năng thu nhận dữ liệu, xử lý theo thời gian thực, giao tiếp và cấu hình lại. Các xu hướng chính gồm kích thước nhỏ hơn, giảm điện năng tiêu thụ, khả năng tính toán và kết nối tốt hơn, giao thức được chuẩn hóa hơn. Đây là *xu thế thiết kế chung*; không phải tất cả các thiết bị đời mới luôn tiết kiệm năng lượng hơn trong mọi điều kiện hoạt động.

Theo bài giảng Chapter 1: An overview of IoT, trang PDF 15–17.

1.3.3 Smart Environment (SE) – môi trường thông minh

Môi trường thông minh là không gian vật lý có nhiều đối tượng thông minh được bố trí, kết nối và tương tác với con người hoặc với nhau: nhà thông minh, tòa nhà, bệnh viện, nhà máy, đô thị. Một cảm biến độc lập chưa tạo thành SE; ý nghĩa đến từ khả năng phối hợp thiết bị, xử lý ngữ cảnh và cung cấp dịch vụ hữu ích. Bài giảng liên hệ SE với quan niệm tính toán có mặt khắp nơi của Mark Weiser.

Theo bài giảng Chapter 1: An overview of IoT, trang PDF 18–19.

Kiến thức mở rộng

Ubiquitous computing (Ubicomp) – tính toán khắp nơi: tài nguyên tính toán được tích hợp vào đồ vật và bối cảnh thường ngày, giúp tương tác ít gây chú ý. **IoT** nhấn mạnh thêm *kết nối và trao đổi thông tin*. Một thiết bị cảm biến đèn tự bật sáng cục bộ có thể thuộc khái niệm hệ thống nhúng/Ubicomp nhưng chưa chắc là IoT; khi nối với nền tảng giám sát/quản lý từ xa, ta thấy rõ tính IoT. Cách so sánh này phù hợp tư duy của McEwen–Cassimally (2014, Chương 1).

1.4 Những khái niệm liên quan: M2M, IIoT, Industry 4.0

1.4.1 Machine-to-Machine (M2M)

M2M là truyền thông giữa các máy/thiết bị mà không cần con người can thiệp trực tiếp vào từng trao đổi. Ví dụ máy đo điện gửi chỉ số về bộ tập trung; bộ điều khiển công nghiệp gửi trạng thái cho máy giám sát. M2M và IoT có vùng giao nhau: M2M tập trung vào các kênh máy–máy phục vụ tác vụ; IoT thường đặt chúng vào hệ thống dịch vụ, dữ liệu, quản lý thiết bị và ứng dụng quy mô rộng hơn. **Không đồng nhất M2M với Internet**: M2M có thể chạy trên mạng riêng hoặc liên kết trực tiếp.

1.4.2 Industrial Internet of Things (IIoT)

IIoT là việc vận dụng IoT trong hoạt động công nghiệp, nơi dữ liệu được dùng để theo dõi máy móc, chất lượng quy trình, tình trạng thiết bị, an toàn và bảo trì. IIoT thường chú trọng độ tin cậy, thời gian phản hồi, tính sẵn sàng và tích hợp hệ thống điều khiển hiện có.

1.4.3 Industry 4.0 – công nghiệp 4.0

Industry 4.0 là định hướng chuyển đổi sản xuất với sự kết hợp của hệ thống cyber–physical, kết nối dữ liệu, tự động hóa và phân tích. **IIoT là một nhóm công nghệ nền tảng quan trọng, nhưng không bao trùm toàn bộ Industry 4.0**. Ví dụ AI kiểm soát chất lượng, robot cộng tác, mô hình song sinh số và hệ hoạch định sản xuất có thể đi cùng IIoT.

Theo bài giảng Chapter 1: An overview of IoT, trang PDF 20–22.

Bảng 1.2: So sánh các khái niệm dễ nhầm lẫn

Khái niệm	Trọng tâm	Điều kiện đặc trưng	Ví dụ
Smart Object	Từng đối tượng	Cảm nhận, xử lý, kết nối hoặc tác động	Đồng hồ điện thông minh
Smart Environment	Toàn không gian	Nhiều thiết bị và dịch vụ phối hợp	Tòa nhà thông minh
Ubicomp	Tính toán ở mọi nơi	Tính toán ẩn trong đồ vật	Đèn tự bật theo chuyển động
M2M	Giao tiếp máy–máy	Trao đổi dữ liệu tự động	Bộ đo đến bộ điều khiển
IIoT	IoT trong công nghiệp	Đo lường, giám sát, cải thiện vận hành	Giám sát máy nén khí
Industry 4.0	Chuyển đổi sản xuất	Tích hợp tự động hóa, dữ liệu, tổ chức	Sản xuất thông minh

1.5 Tư duy hệ thống: từ dữ liệu tới hành động

Trong một quy trình IoT hoàn chỉnh, đầu vào là các đại lượng vật lý: nhiệt độ, áp suất, rung, ánh sáng, vị trí, trạng thái công tắc. **Cảm biến** biến đổi đại lượng cần quan sát thành tín hiệu; bộ xử lý đưa dữ liệu về dạng có thể dùng (số hóa, lọc, hiệu chuẩn); lớp kết nối chuyển dữ liệu; nền tảng phân tích áp dụng quy tắc; ứng dụng hiển thị thông tin hoặc phát lệnh; cơ cấu chấp hành thực hiện hành động. Kỹ sư cần xác định rõ *lấy số đo gì, để giải quyết quyết định nào, cho ai, trong điều kiện vận hành nào.*

Kiến thức mở rộng

Ví dụ hoàn chỉnh: hệ tưới cây. Cảm biến đo độ ẩm đất theo chu kỳ; MCU lọc nhiễu và nhận biết vượt ngưỡng; Wi-Fi truyền dữ liệu lên server; dashboard lưu đồ thị; bộ điều khiển mở van tưới và tự đóng khi đạt ngưỡng. Cần thiết kế chế độ an toàn khi mất mạng (ví dụ không tưới liên tục) và cơ chế xác định cảm biến bị lỗi. Đây là ví dụ mở rộng, không có sơ đồ đấu dây trong slide gốc.

1.6 Tổng kết và câu hỏi tự kiểm tra

Ghi nhớ

Nhớ 5 ý: (1) IoT gắn dữ liệu với thế giới vật lý; (2) smart object là nền tảng thiết bị; (3) smart environment là hệ phối hợp; (4) M2M khác phạm vi IoT; (5) IoT là sự hội tụ của phần cứng nhúng, cảm biến, mạng, dịch vụ và dữ liệu.

1. Phân biệt thiết bị nhúng tự động và hệ thống IoT bằng hai tình huống cụ thể.
2. Vì sao thuật ngữ IoT gắn với Kevin Ashton và RFID năm 1999?
3. Nêu bốn giai đoạn phát triển Internet theo slide và giải thích mục tiêu của mỗi giai đoạn.
4. Mô tả năm khối chính của smart object. Thành phần nào không bắt buộc với mọi thiết bị?
5. Nêu một ví dụ về smart environment và giải thích lợi ích của việc phối hợp nhiều thiết bị.
6. So sánh M2M, IIoT, Industry 4.0; cho một trường hợp có cả ba.

CHƯƠNG 2

Thành phần hệ thống và các mô hình kiến trúc IoT

Ghi nhớ

Mục tiêu: nhận diện vai trò của Things, Gateway, Cloud, Services và giao diện người dùng; phân biệt mô hình 3, 4, 5, 6 và 7 lớp trong bài giảng; hiểu luồng dữ liệu và điều khiển hai chiều; biết lựa chọn mô hình phù hợp với yêu cầu hệ thống.

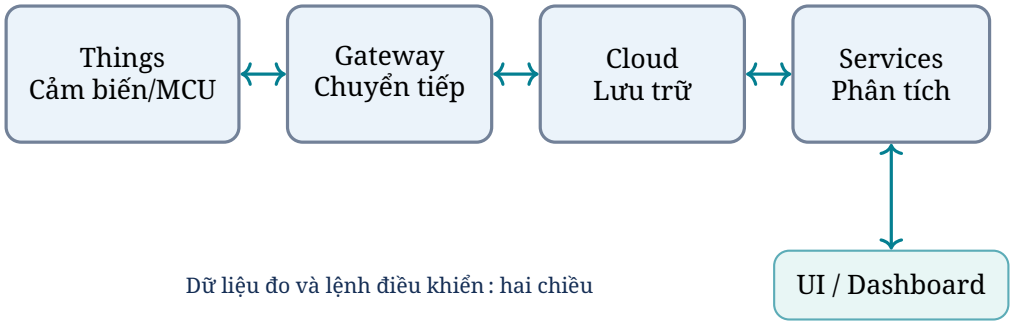
2.1 Tư duy thiết kế theo kiến trúc tổng thể

Kiến trúc IoT mô tả cách các thành phần được phân nhóm, kết nối, trao đổi dữ liệu và cung cấp dịch vụ. Kiến trúc giúp trả lời: dữ liệu sinh ra ở đâu, truyền qua đường nào, xử lý tại đâu, được lưu như thế nào, ai được phép truy cập và khi nào phải thực hiện lệnh điều khiển. Các kiến trúc 3/4/5/6/7 lớp là **mô hình nhìn ở những độ chi tiết khác nhau**, không nhất thiết là năm thế hệ kế tiếp thay thế nhau.

2.2 Bốn khối chức năng và giao diện người dùng

Bài giảng giới thiệu bốn khối **Things** – **Gateway/Network** – **Cloud** – **Services** và ngay sau đó mô tả thêm **User Interface (UI)**. Khi vẽ hệ thống, nên đặt UI như lớp tương tác bên ngoài bốn khối để không làm mất chi tiết này.

Theo bài giảng Chapter 1: An overview of IoT, trang PDF 23–28.



Hình 2.1: Các khối chức năng của hệ thống IoT được bài giảng giới thiệu.

2.2.1 Things/Devices – thiết bị đầu cuối

Things có thể chứa cảm biến, cơ cấu chấp hành, bộ điều khiển hoặc bộ xử lý như MCU, MPU, PLC, FPGA, máy tính nhúng. Nhiệm vụ: đo lường, thu thập dữ liệu, tiền xử lý, thực thi logic, phản hồi lệnh và trao đổi dữ liệu. Cảm biến thường gặp: nhiệt độ, áp suất, độ ẩm, gia tốc/chuyển động, ánh sáng, mức chất lỏng, khói. Không phải mọi node đều có cả cảm biến lẫn actuator; nút đo có thể chỉ truyền, nút điều khiển có thể chỉ nhận lệnh.

Bảng kiểm ban đầu của một nút: *đại lượng đo, dải đo, sai số, tốc độ lấy mẫu, khả năng xử lý, bộ nhớ, nguồn, thời gian hoạt động, phương thức kết nối, vỏ bảo vệ, khả năng cập nhật và khôi phục*. Trong bản thiết kế nghiêm túc cần có thông số điện áp, công suất đỉnh, chu kỳ phát dữ liệu và trạng thái lỗi.

Theo bài giảng Chapter 1: An overview of IoT, trang PDF 24.

2.2.2 Gateway/Network – cổng kết nối và liên mạng

Gateway tiếp nhận nhiều nút cục bộ, định tuyến hoặc chuyển tiếp thông điệp, chuyển đổi giao thức và có thể **tiền xử lý (pre-processing)** như lọc giá trị không hợp lệ, gộp mẫu, nén dữ liệu, bổ sung dấu thời gian. Thiết bị gateway còn có thể thực hiện kiểm tra xác thực, mã hóa hoặc quản lý truy cập, nhưng **không thể mặc định gateway tự bảo đảm toàn bộ an ninh hệ thống**. Thiết kế phải tính bảo mật thiết bị, liên kết truyền và dịch vụ đầu cuối.

Ví dụ, nhiều cảm biến dùng BLE hoặc mạng vô tuyến cục bộ truyền tới Raspberry Pi; gateway dùng Ethernet/4G để chuyển dữ liệu đến máy chủ qua giao thức IP. Cần phân biệt *gateway* (chuyển đổi/trung gian) và *router* (định tuyến): đôi khi một thiết bị kiêm cả hai, nhưng chức năng không đồng nhất.

Theo bài giảng Chapter 1: An overview of IoT, trang PDF 25.

2.2.3 Cloud – hạ tầng lưu trữ và tính toán

Cloud cung cấp năng lực tiếp nhận, quản lý, lưu trữ và phân tích dữ liệu từ số lượng lớn thiết bị. Những chức năng thường có: quản lý định danh và vòng đời thiết bị, máy chủ tiếp nhận, hàng đợi thông điệp, cơ sở dữ liệu chuỗi thời gian, công cụ xử lý, cảnh báo, phân quyền và giám sát hoạt động. Chia tách xử lý thời gian thực với thống kê/lưu trữ dài hạn giúp hệ thống dễ mở rộng.

Theo bài giảng Chapter 1: An overview of IoT, trang PDF 26.

2.2.4 Services – dịch vụ và phân tích nghiệp vụ

Services chuyển dữ liệu đã thu thập thành **thông tin có thể ra quyết định**: hiển thị trạng thái; phân tích xu hướng; phát hiện bất thường; dự báo bảo trì; gửi cảnh báo; tối ưu vận hành. Cần phân biệt **data** (số đo) và **information/insight** (kết quả diễn giải): dãy giá trị nhiệt độ là dữ liệu, cảnh báo máy có nguy cơ quá nhiệt là thông tin phục vụ hành động.

Theo bài giảng Chapter 1: An overview of IoT, trang PDF 27.

2.2.5 User Interface (UI) – giao diện người dùng

UI giúp người vận hành xem và điều khiển hệ thống: dashboard trên web, mobile app, màn hình HMI hoặc công cụ bảo trì. UI nên hiển thị thời điểm cập nhật, đơn vị đo, trạng thái liên lạc, mức cảnh báo, quyền truy cập và lịch sử thao tác. **Thiết kế giao diện không chỉ là hình thức**: ở IoT, UI làm rõ tình trạng thiết bị và giảm nguy cơ thao tác sai.

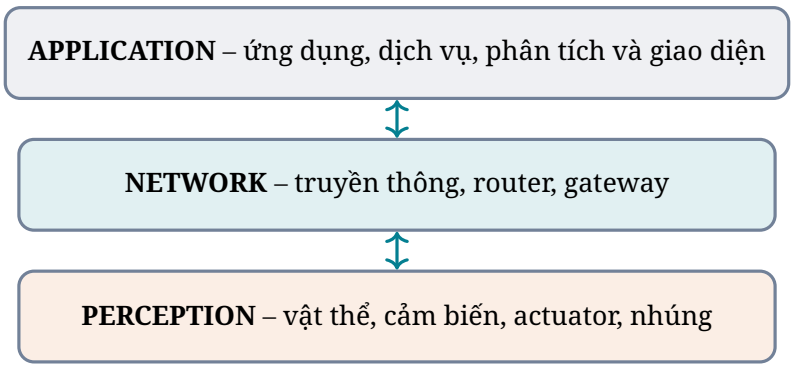
Theo bài giảng Chapter 1: An overview of IoT, trang PDF 28.

Kiến thức mở rộng

Đường đi dữ liệu mẫu: cảm biến áp suất → giao tiếp I^2C → MCU → gói dữ liệu → gateway → MQTT/HTTPS → cơ sở dữ liệu → dashboard. Lệnh cấu hình ngưỡng có thể đi ngược hướng về MCU. Các giao thức này là *ví dụ mở rộng*; bài giảng phần kiến trúc không chỉ định một giao thức bắt buộc.

2.3 Kiến trúc 3 lớp (three-layer architecture)

Mô hình giản lược **Perception** – **Network** – **Application** là cách phân tách phổ biến để giới thiệu hệ thống.



Hình 2.2: Kiến trúc IoT ba lớp; các mũi tên chỉ trao đổi hai chiều.

2.3.1 Perception layer – lớp cảm nhận

Bao gồm **hardware** (phần cứng nền), **IoT components** (cảm biến, actuator, bộ xử lý, antenna/giao tiếp) và **device software** (firmware để quản lý phần cứng). Nhiệm vụ nhận diện đối tượng, đo lường và xuất dữ liệu từ môi trường. Các lỗi thường gặp: sai số cảm biến, nhiễu, mất mẫu, cạn pin, sai đơn vị hoặc sai điểm hiệu chuẩn.

2.3.2 Network layer – lớp mạng

Chịu trách nhiệm vận chuyển dữ liệu giữa things, hạ tầng kết nối và/hoặc đám mây. Thành phần có thể dùng Wi-Fi, BLE, mạng di động, Ethernet, các liên kết vô tuyến có gateway. Vấn đề cần khảo sát: độ phủ sóng, độ trễ, thông lượng, tỷ lệ mất gói, bảo mật và khả năng phục hồi khi mất kết nối.

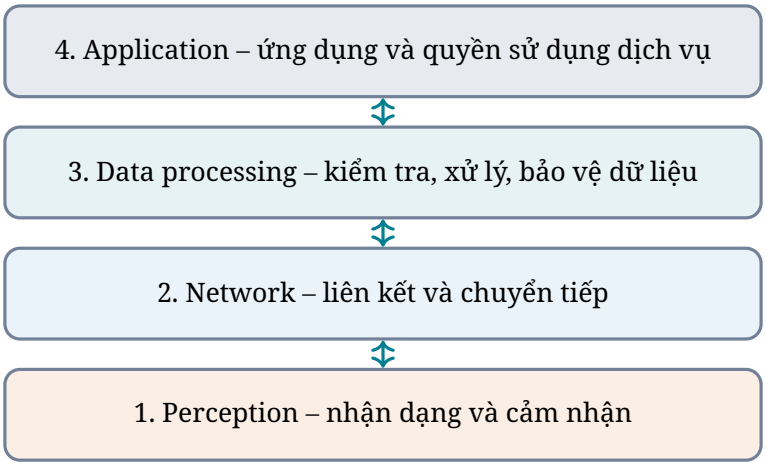
2.3.3 Application layer – lớp ứng dụng

Là nơi cung cấp chức năng theo mục tiêu người dùng: trao đổi/quản lý thiết bị, ứng dụng, công cụ phân tích dữ liệu, quản lý quy trình và nền tảng phát triển. Slide 33 phân tách nhiều chức năng phần mềm ngay trong lớp ứng dụng; cần nhớ **lớp ứng dụng không chỉ là một ứng dụng điện thoại**.

Theo bài giảng Chapter 1: An overview of IoT, trang PDF 29–33.

2.4 Kiến trúc 4 lớp hướng dịch vụ (SoA)

Service-Oriented Architecture (SoA) là tư duy xây dựng các khối chức năng độc lập tương đối, liên kết bằng giao diện và giao thức. Bài giảng trình bày kiến trúc bốn lớp: **Perception** – **Network** – **Data Processing** – **Application**. Lớp xử lý dữ liệu được tách riêng để quản lý, kiểm tra, xử lý thông tin trước khi cung cấp lên ứng dụng.



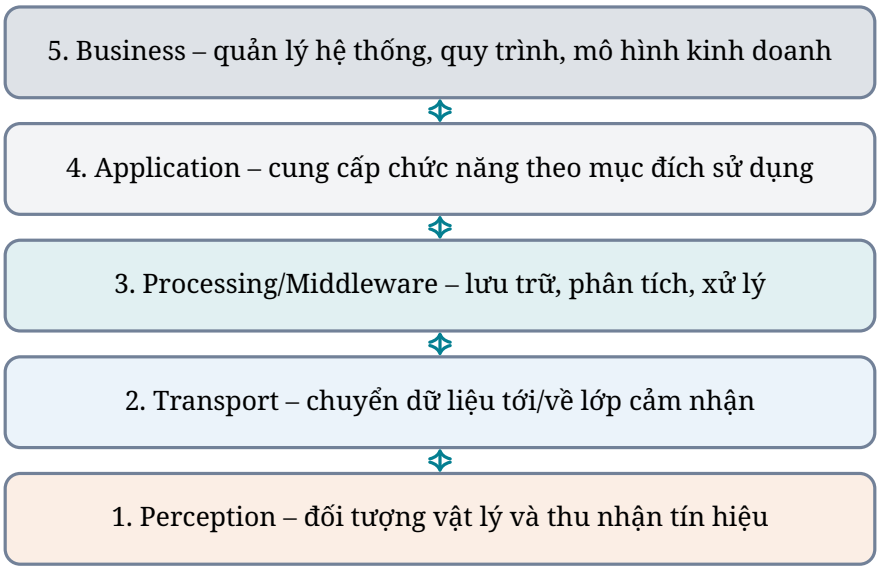
Hình 2.3: Mô hình bốn lớp hướng dịch vụ theo bài giảng.

Bài giảng nhấn mạnh: application cung cấp giao diện và dịch vụ; data processing tiếp nhận dữ liệu từ perception thông qua network, kiểm tra tính hợp lệ và xử lý; network vận chuyển và liên kết mạng; perception nhận biết từng loại cảm biến và tập hợp số đo. Lưu ý **"hướng dịch vụ" là nguyên tắc tổ chức chức năng**, không phải tên một giao thức truyền thông như MQTT, HTTP.

Theo bài giảng Chapter 1: An overview of IoT, trang PDF 34–36.

2.5 Kiến trúc 5 lớp (five-layer architecture)

Mô hình năm lớp được diễn giải **Perception** – **Transport** – **Processing/Middleware** – **Application** – **Business**. So với mô hình ba lớp, lớp vận chuyển được định danh riêng và thêm các khối xử lý, quản trị nghiệp vụ.



Hình 2.4: Cấu trúc năm lớp; xử lý dữ liệu và quản lý nghiệp vụ được phân tách.

Perception
Cảm nhận, định danh đối tượng, lấy dữ liệu. Bao gồm RFID, barcode và các loại cảm biến trong mô tả mở rộng của bài giảng.

Transport
Truyền dữ liệu qua mạng có dây, mạng không dây, mạng di động, Bluetooth, RFID, NFC và hệ thống trung gian; liệt kê này biểu thị các *phương tiện và công nghệ*, không phải tất cả nằm cùng một lớp giao thức OSI.

Processing/Middleware
Cơ sở dữ liệu, điện toán đám mây, xử lý dữ liệu lớn, quản lý dịch vụ, thuật toán ra quyết định.

Application
Phần mềm người dùng và hệ thống chức năng, ví dụ giám sát nhiệt độ hoặc quản lý tòa nhà.

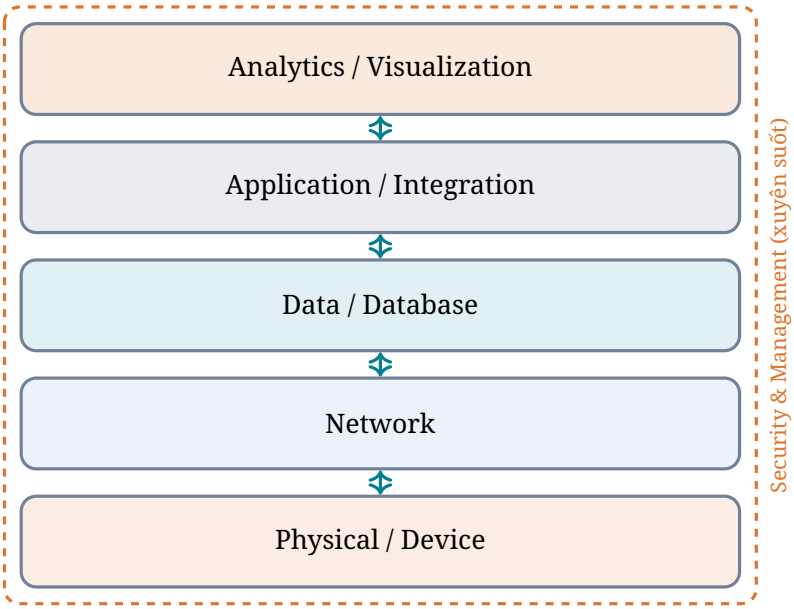
Business
Quản lý mục tiêu và hiệu quả hệ thống, mô hình kinh doanh, chính sách và quyền riêng tư của người dùng.

Theo bài giảng Chapter 1: An overview of IoT, trang PDF 37–39.

2.6 Kiến trúc 6 và 7 lớp trong hình của bài giảng

2.6.1 Sáu lớp: chú ý cách đếm trong chính slide

Trang PDF 40 có tiêu đề *Six-Layer IoT Architecture*, nhưng hình thực tế vẽ **năm dải chức năng theo chiều dọc** gồm physical/device, network, data/database, application/integration, analytics/visualization; bên cạnh là **security and management layer** chạy xuyên suốt. Vì vậy trong tài liệu này, ta hiểu đây là **năm lớp chức năng cộng một lớp/chức năng xuyên suốt bảo mật–quản trị**. Đây là cách đọc trung thành với hình chứ không ép biến bảo mật thành một tầng truyền dữ liệu độc lập.

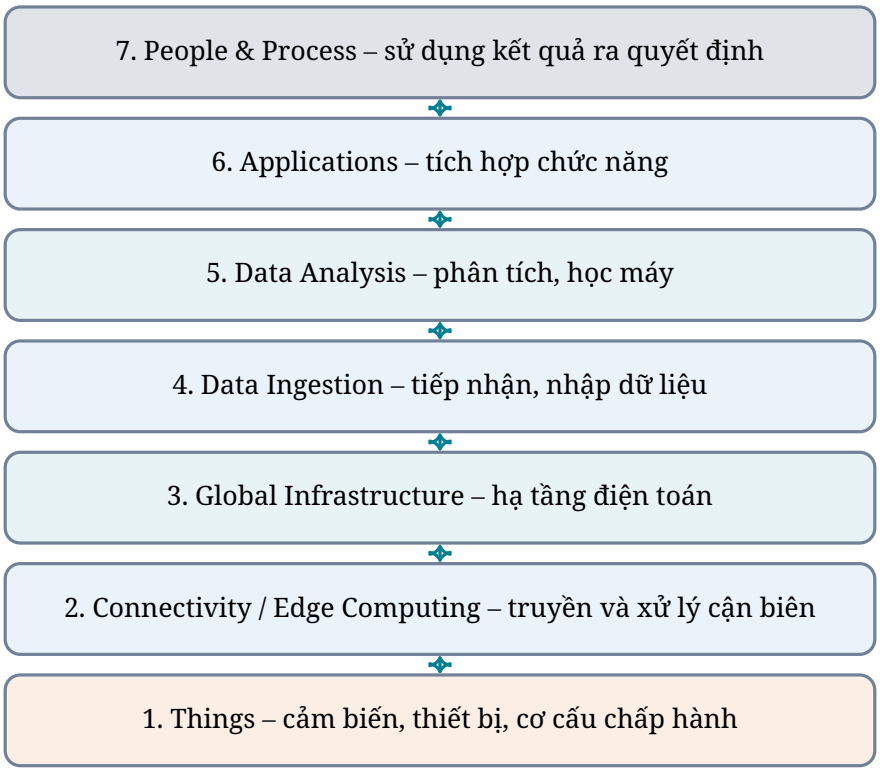


Hình 2.5: Diễn giải hình sáu lớp: năm dải chức năng và bảo mật–quản trị xuyên suốt.

2.6.2 Bảy lớp: dữ liệu và giá trị được tách chi tiết hơn

Hình kiến trúc 7 lớp (trang PDF 41) đưa ra chuỗi:

1. **Things**: thiết bị, cảm biến, bộ điều khiển.
2. **Connectivity/Edge Computing**: giao thức, mạng, xử lý gần nguồn dữ liệu.
3. **Global Infrastructure**: hạ tầng công cộng/riêng/hỗn hợp cung cấp tính toán và kết nối.
4. **Data Ingestion**: nhận và tiếp nhận khối dữ liệu.
5. **Data Analysis**: báo cáo, khai phá dữ liệu, học máy.
6. **Applications**: biến dữ liệu thành chức năng sử dụng.
7. **People & Process**: con người, quy trình và tác động kinh doanh.



Hình 2.6: Tái dựng kiến trúc bảy lớp theo thành phần mô tả trong slide.

Theo bài giảng Chapter 1: An overview of IoT, trang PDF 40–41.

2.7 Bảng đối chiếu nhanh các mô hình

Bảng 2.1: Cách đọc các mô hình kiến trúc trong bài giảng

Mô hình	Phân chia chính	Điểm cần nhớ
3 lớp	Perception – Network – Application	Đơn giản, dễ nắm đường đi dữ liệu.
4 lớp	Perception – Network – Data Processing – Application	Chú ý lớp xử lý và tư duy SoA.
5 lớp	Perception – Transport – Processing – Application – Business	Tách quản lý nghiệp vụ ra khỏi ứng dụng.
6 lớp	5 lớp dọc, Security & Management ngang	Lưu ý cách thể hiện ở slide, bảo mật xuyên suốt.
7 lớp	Things – Connectivity/Edge – Global Infrastructure – Data Ingestion – Data Analysis – Applications – People/Process	Theo sát đường dữ liệu từ thiết bị tới giá trị nghiệp vụ.

2.8 Ví dụ đặt các thành phần vào từng lớp

Xét hệ thống quan trắc chất lượng không khí gồm cảm biến PM2.5, nhiệt độ và độ ẩm, MCU, bộ thu trung gian, máy chủ, dashboard và quy tắc báo động. Theo mô hình ba lớp: cảm biến+MCU thuộc perception; bộ thu và đường truyền thuộc network; máy chủ, phân tích và dashboard thuộc application. Theo năm lớp: các nút đo thuộc perception; đường truyền

thuộc transport; database và logic phân tích thuộc processing; dashboard thuộc application; quy trình quản lý chất lượng môi trường và phân quyền vận hành thuộc business.

Kiến thức mở rộng

Thiết kế an toàn theo luồng dữ liệu: đo tại MCU → kiểm tra giá trị và timestamp → đóng gói thông điệp có `device_id` → giao thức truyền có xác thực → tiếp nhận, kiểm tra định dạng và phân quyền → lưu/hiển thị → phát lệnh khi được phép. Đây là ví dụ triển khai tăng cường từ nguyên lý, không phải cấu hình bắt buộc của slide.

2.9 Các chỉ tiêu thiết kế cần đánh giá

2.9.1 Thông lượng và tần suất gửi

Giả sử mỗi mẫu có B byte và được gửi mỗi T giây, tốc độ dữ liệu trung bình theo lý thuyết là

$$R = \frac{8B}{T} \quad \text{bit/s}.$$

Nếu $B = 100$ byte, $T = 10$ giây, $R = 80$ bit/s cho một nút, chưa tính overhead giao thức, truyền lại và bảo mật. Với N nút, lưu lượng tải trung bình lý tưởng xấp xỉ NR; thực tế phải cộng chi phí truyền thông và tỷ lệ nút gửi đồng thời.

2.9.2 Độ trễ, độ tin cậy và tính khả dụng

Latency là thời gian từ sự kiện đến phản hồi; **reliability** là xác suất hoạt động chính xác trong điều kiện xác định; **availability** là tỷ lệ thời gian dịch vụ sẵn sàng; **fault tolerance** là khả năng tiếp tục cung cấp chức năng khi một số thành phần gặp lỗi. Một hệ điều khiển cảnh báo tại chỗ không nên phụ thuộc hoàn toàn vào độ trễ cloud.

2.9.3 Năng lượng của nút IoT

Với chế độ hoạt động i, dòng tiêu thụ I_i và tỷ lệ thời gian d_i, dòng trung bình ước lượng là

$$I_{\text{avg}} = \sum_i d_i I_i, \qquad \sum_i d_i = 1.$$

Ước lượng thời gian dùng pin lý tưởng là $t \approx C_{\text{usable}}/I_{\text{avg}}$ (giờ), khi C đo bằng mAh và I đo bằng mA. Để sử dụng công thức phải xét tổn hao bộ nguồn, xung dòng đỉnh, nhiệt độ, độ lão hóa và giới hạn điện áp. Các công thức này là **kiến thức mở rộng** hỗ trợ mục tiêu giảm tiêu thụ năng lượng nêu ở slide.

2.10 Tổng kết và câu hỏi tự kiểm tra

Ghi nhớ

Nhớ 6 ý: Things tạo dữ liệu; Gateway chuyển/tiền xử lý; Cloud xử lý và lưu trữ quy mô lớn; Services diễn giải để phục vụ quyết định; UI kết nối hệ thống với người dùng; phân lớp khác nhau giúp đặt đúng trách nhiệm kỹ thuật.

1. Vẽ kiến trúc chức năng bốn khối cộng UI, nêu dữ liệu và lệnh đi theo các hướng nào.

2. Vì sao gateway không nhất thiết giống router? Hãy nêu ba chức năng có thể bổ sung vào gateway.
3. Mô tả ba lớp của mô hình IoT cơ bản và chỉ ra firmware thuộc lớp nào.
4. So sánh bốn lớp SoA với năm lớp: có thêm hoặc thay đổi khối nào?
5. Vì sao trong slide sáu lớp, security and management cần được hiểu là chức năng xuyên suốt?
6. Liệt kê bảy lớp theo sơ đồ và giải thích hai lớp liên quan tới xử lý dữ liệu.
7. Một mạng có 50 nút, mỗi nút gửi 120 byte mỗi 20 giây. Tính lưu lượng hữu ích trung bình (chưa kể overhead). Đáp số: $50 \times 120 \times 8/20 = 2400$ bit/s.

CHƯƠNG 3

Ứng dụng, cơ hội và thách thức của Internet of Things

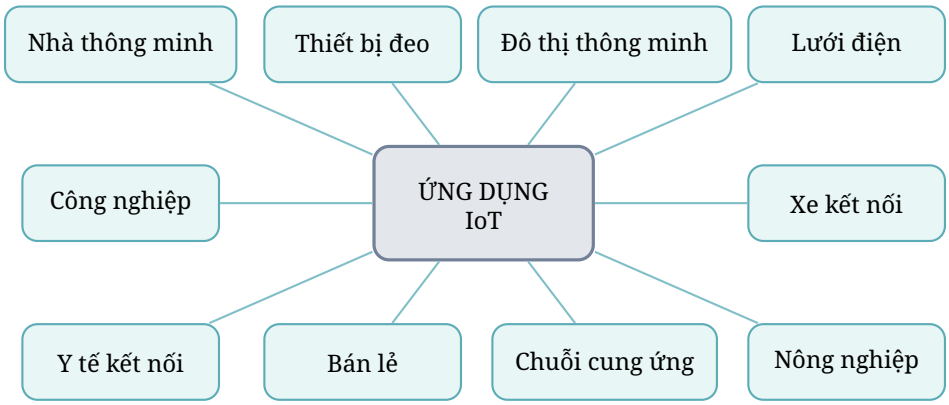
Ghi nhớ

Mục tiêu: nhận biết các nhóm ứng dụng IoT; phân tích hai tình huống trong bài giảng (nhà thông minh và ô tô kết nối); hiểu chuỗi giá trị từ dữ liệu tới dịch vụ; trình bày các thách thức kỹ thuật, xã hội, đạo đức và vận hành khi triển khai.

3.1 Các lĩnh vực ứng dụng IoT

Bài giảng sử dụng hình tổng quan với các ví dụ: **smart home**, **wearables**, **smart city**, **smart grid**, **industrial internet**, **connected car**, **connected health**, **smart retail**, **smart supply chain**, **smart farming**. Đây là những lĩnh vực ứng dụng, không phải mười giao thức hay mười kiểu kiến trúc riêng biệt.

Theo bài giảng Chapter 1: An overview of IoT, trang PDF 42–43.



Hình 3.1: Mười nhóm ứng dụng liệt kê ở phần Internet of Things Applications của bài giảng.

3.1.1 Nhà thông minh và tòa nhà thông minh

Các cảm biến nhiệt độ, độ ẩm, chuyển động, ánh sáng, cửa và công suất điện cùng thiết bị điều khiển đèn, máy lạnh, khóa, ổ cắm, rèm tạo thành hệ thống phối hợp. Giá trị nằm ở tự động hóa theo ngữ cảnh (ví dụ tắt thiết bị khi không có người), giám sát từ xa và tiết kiệm năng lượng. Rủi ro: mất kết nối, truy cập trái phép và phụ thuộc nhà cung cấp.

3.1.2 Thiết bị đeo và y tế kết nối

Thiết bị đeo theo dõi vận động, nhịp tim hoặc điều kiện môi trường; hệ chăm sóc sức khỏe có thể tổng hợp dữ liệu để hỗ trợ theo dõi từ xa. Cần chú trọng chất lượng số đo, bảo vệ dữ liệu cá nhân, cảnh báo giả và giới hạn của thiết bị không đạt chứng nhận y tế. **Một thiết bị theo dõi sức khỏe thông thường không tự động là thiết bị chẩn đoán lâm sàng.**

3.1.3 Đô thị thông minh và giao thông

Cảm biến bãi đỗ, đo môi trường, chiếu sáng công cộng và giám sát giao thông có thể giúp tối ưu tài nguyên đô thị. Xe kết nối có thể thông báo bảo dưỡng, trạng thái động cơ, vị trí và lịch sử vận hành; những ứng dụng yêu cầu phản hồi an toàn không nên dựa duy nhất vào đường truyền Internet không ổn định.

3.1.4 Công nghiệp, điện và chuỗi cung ứng

IIoT thu thập rung/nhiệt độ/dòng điện của máy để phát hiện bất thường, phục vụ bảo trì dựa trên tình trạng. Smart grid hỗ trợ đo điện, điều phối sử dụng, phát hiện bất thường. Smart supply chain kết hợp nhãn nhận dạng, vị trí và trạng thái hàng hóa để theo dõi nguồn gốc và tối ưu logistics.

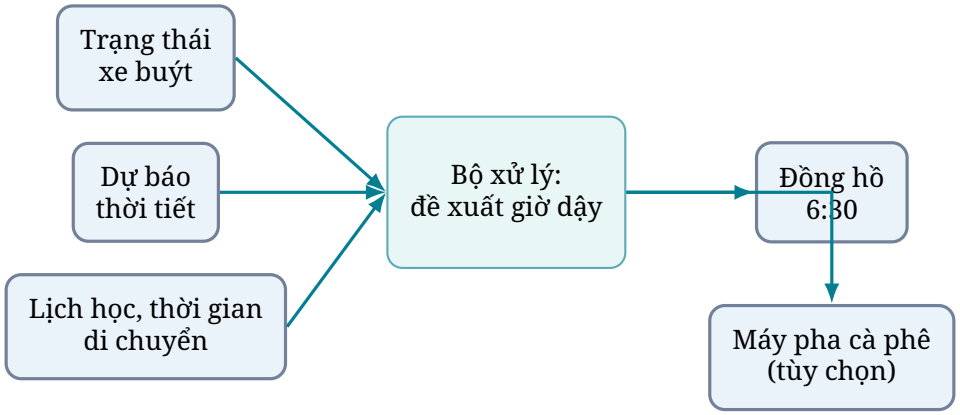
3.1.5 Bán lẻ và nông nghiệp thông minh

Bán lẻ: theo dõi hàng tồn kho, mức sử dụng thiết bị làm lạnh, điều kiện bảo quản. Nông nghiệp: quan trắc độ ẩm đất, thời tiết cục bộ, dinh dưỡng, tình trạng cây trồng, kích hoạt tưới tiêu có điều kiện. IoT có thể tạo dữ liệu hữu ích nhưng không thay thế hoàn toàn quy trình kiểm tra thực địa.

3.2 Tình huống 1 trong bài giảng: đồng hồ báo thức thông minh

Bài giảng trình bày một người đi học thường dậy lúc **7:00**. Khi xe buýt bị hủy, trời mưa khiến phương án đi Grab không khả thi và phải di chuyển bằng xe đạp, người đó cần dậy lúc **6:30**. Đồng hồ báo thức IoT có thể lấy thông tin lịch xe, thời tiết và thời gian di chuyển để tự điều chỉnh giờ thức dậy. Nếu kết nối máy pha cà phê, hệ thống còn có thể chuẩn bị đồ uống theo giờ thức dậy mới.

Theo bài giảng Chapter 1: An overview of IoT, trang PDF 44.



Hình 3.2: Tái dựng luồng ra quyết định trong tình huống đồng hồ báo thức.

Phân tích theo khối: dữ liệu ngoài (xe buýt, thời tiết) → dịch vụ ra quyết định → lệnh cấu hình báo thức và thiết bị trong nhà. Giá trị cốt lõi không phải chỉ "có Wi-Fi", mà là khả năng thích ứng với ngữ cảnh thực tế để đạt mục tiêu đến lớp đúng giờ.

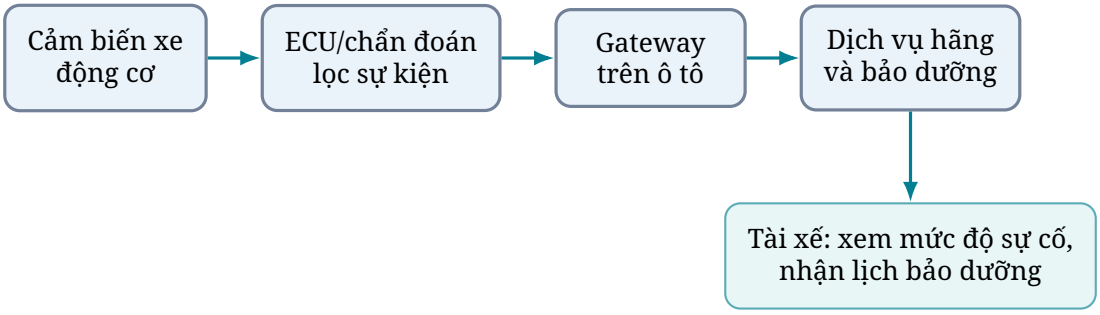
Kiến thức mở rộng

Những vấn đề thiết kế cần giải quyết: dữ liệu bên thứ ba có thể chậm hoặc sai; dự báo thời tiết không chắc chắn; điện thoại và báo thức phải đồng bộ múi giờ; hệ thống phải có giờ báo thức dự phòng khi mất mạng; người dùng phải có quyền từ chối thay đổi tự động. Đây là những tiêu chí mở rộng từ tình huống trong slide.

3.3 Tình huống 2 trong bài giảng: ô tô kết nối

Khi đèn cảnh báo bảo dưỡng động cơ bật sáng, các bộ cảm biến trên ô tô cung cấp dữ liệu liên quan. Một cơ chế chẩn đoán tổng hợp thông tin rồi gửi tới gateway trên xe; gateway chuyển thông tin phù hợp tới nhà sản xuất/đơn vị dịch vụ. Dữ liệu có thể giúp đánh giá mức khẩn cấp, đề xuất cơ sở sửa chữa gần nhất, đặt lịch và chuẩn bị phụ tùng.

Theo bài giảng Chapter 1: An overview of IoT, trang PDF 45.



Hình 3.3: Chuỗi chẩn đoán và dịch vụ bảo dưỡng của ô tô kết nối.

Tách vai trò: cảm biến trả lời "đang xảy ra điều gì"; ECU và bộ chẩn đoán trả lời "hiện tượng có nghĩa gì"; gateway chuyển dữ liệu; nền tảng dịch vụ quyết định thông báo hoặc sắp lịch; người điều khiển/nhân viên bảo dưỡng thực hiện hành động. Từ góc nhìn kinh doanh, hệ thống chuyển từ sửa chữa khi hỏng sang chủ động tiếp cận dịch vụ.

3.4 Cơ hội do IoT mang lại

Slide về **IoT Challenges and Opportunities** nhấn mạnh bốn nhóm tác động: *New Markets*, *Changed Value Creation*, *Increased Awareness for Information Spaces*, và *Social, Ethical, Legal and Risk Aspects*. Ba nhóm đầu diễn đạt cơ hội; nhóm cuối đặt ra yêu cầu cân nhắc trách nhiệm đối với dữ liệu và người sử dụng.

Theo bài giảng Chapter 1: An overview of IoT, trang PDF 47.

3.4.1 Thị trường mới (New Markets)

Sự kết nối tạo ra sản phẩm, dịch vụ và thị trường mới: giám sát môi trường theo thuê bao, quản lý thiết bị từ xa, bảo trì dự báo, phân tích sử dụng tài sản. Một sản phẩm vật lý có thể được bổ sung dịch vụ số; doanh nghiệp phải tính cả vận hành máy chủ, bảo trì firmware và hỗ trợ khách hàng sau khi bán.

3.4.2 Thay đổi cách tạo giá trị (Changed Value Creation)

Giá trị không chỉ ở phần cứng mà ở chất lượng dữ liệu, việc phân tích và tối ưu quy trình. Ví dụ cảm biến nhiệt độ giá thấp tự nó không tạo lợi ích lớn; khi gắn với cảnh báo bảo quản vaccine hoặc thực phẩm, hệ thống có thể ngăn thiệt hại và hỗ trợ ra quyết định.

3.4.3 Nhận biết rõ hơn không gian thông tin

Dữ liệu đo từ các vật thể làm cho trạng thái của môi trường trở nên quan sát được: vị trí xe, điện năng tiêu thụ, độ rung máy, mức nước, xu hướng chất lượng không khí. Tuy nhiên tăng lượng dữ liệu **không đồng nghĩa tự động tăng chất lượng thông tin**: phải đánh giá nguồn đo, sai số, hiệu chuẩn, độ trễ và ngữ cảnh.

3.5 Các thách thức công nghệ trong triển khai IoT

Bài giảng nêu: Internet mọi nơi/mọi lúc; khả năng chấp nhận xã hội; nút cảm biến giá rẻ; thu thập năng lượng; năng lực tính toán; bảo mật; mở rộng quy mô, độ tin cậy; chịu lỗi; mức tiêu thụ điện của node và transceiver. Các mục dưới đây giải thích từng nhóm theo câu hỏi *vì sao khó* và *giải quyết ở cấp thiết kế nào*.

Theo bài giảng Chapter 1: An overview of IoT, trang PDF 48.

3.5.1 Kết nối mọi nơi – mạng không luôn sẵn sàng

Thiết bị có thể lắp dưới hầm, trong nhà máy nhiều nhiễu hoặc vùng ít hạ tầng. Các mạng khác nhau có giới hạn về tầm phủ, tốc độ, chi phí, năng lượng và độ trễ. Giải pháp về mặt nguyên tắc: lựa chọn công nghệ kết nối đúng yêu cầu, có bộ đệm khi offline, truyền lại khi phù hợp, thiết kế cơ chế cảnh báo trạng thái mất kết nối và xử lý an toàn tại chỗ.

3.5.2 Chi phí thấp và giới hạn tài nguyên phần cứng

Nút cảm biến thường có RAM/flash nhỏ, CPU hạn chế và ngân sách điện năng khắt khe. Giá thành sản xuất thấp phải đi cùng độ chính xác, độ ổn định và khả năng bảo trì. Cần chọn cảm biến theo dải đo, độ phân giải, giao tiếp, tuổi thọ, độ sẵn có của linh kiện và yêu cầu hiệu chuẩn chứ không chỉ giá thấp nhất.

3.5.3 Năng lượng và power management

Các mô-đun vô tuyến có thể tiêu thụ dòng lớn trong lúc phát; hoạt động sleep tiêu thụ ít hơn nhưng làm tăng thời gian phản hồi. Kỹ sư phải cân bằng chu kỳ đo, chu kỳ truyền, xử lý cục bộ và độ trễ cảnh báo. **Energy harvesting** (thu hồi năng lượng) có thể dùng nguồn môi trường như ánh sáng hoặc rung động, song khả năng cấp điện phụ thuộc điều kiện triển khai.

Kiến thức mở rộng

Ví dụ tính nhanh: một nút sleep 9.9 giây tại 0.05 mA và phát 0.1 giây tại 80 mA mỗi chu kỳ 10 giây. Dòng trung bình lý tưởng

$$I_{\text{avg}} = \frac{9.9 \times 0.05 + 0.1 \times 80}{10} = 0.8495 \text{ mA}.$$

Với dung lượng khả dụng giả định 2000 mAh, thời gian ước lượng lý tưởng $2000/0.8495 \approx 2354$ giờ, gần 98 ngày. Phép tính **không** bao gồm các dòng tiêu thụ khác, hiệu suất nguồn, tự xả hay suy giảm dung lượng; không dùng làm cam kết thời gian pin thực tế.

3.5.4 Scalability – khả năng mở rộng

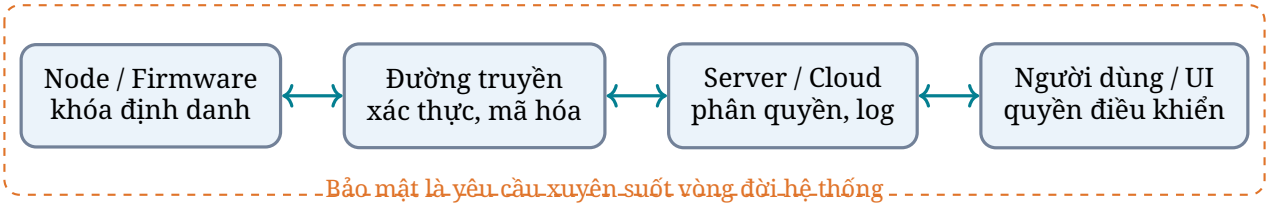
Hệ thống phải chịu được số node, số thông điệp, dung lượng lưu và lượng người dùng tăng lên. Khi gấp đôi số nút, các nút cùng báo cáo tại một thời điểm có thể gây tải đỉnh vượt khả năng gateway hoặc broker. Cần tính tải trung bình và tải đỉnh; phân chia vùng, giới hạn tốc độ, cân bằng tải, phân quyền và lưu trữ theo vòng đời dữ liệu.

3.5.5 Reliability và fault tolerance

Reliability đề cập khả năng cung cấp đúng chức năng; **fault tolerance** đề cập khả năng vẫn hoạt động hoặc suy giảm có kiểm soát khi gặp sự cố. Thiết kế phải xác định trạng thái **fail-safe**: hệ thống tưới mất kết nối không được kẹt van mở liên tục; cảnh báo cháy không nên phụ thuộc duy nhất vào cloud. Bộ nhớ đệm, watchdog, retry có giới hạn và cơ chế phục hồi là các biện pháp có thể cân nhắc.

3.5.6 Security – an ninh hệ thống

Các nguy cơ gồm thiết bị giả mạo, lộ khóa và mật khẩu, nghe lén, sửa gói, firmware độc hại, truy cập dashboard trái phép và quá tải dịch vụ. Bảo mật phải có từ thiết bị tới máy chủ: xác thực, kênh mã hóa phù hợp, nguyên tắc đặc quyền tối thiểu, quản lý khóa, cập nhật an toàn và nhật ký kiểm tra. **Mã hóa đường truyền không thay thế xác thực và phân quyền đầu cuối.**



Hình 3.4: Các lớp kiểm soát an ninh từ thiết bị đến người dùng.

3.6 Thách thức xã hội, đạo đức, pháp lý và rủi ro

3.6.1 Sự chấp nhận của cá nhân và cộng đồng

Người dùng có thể lo ngại thiết bị quá phức tạp, theo dõi liên tục, phụ thuộc Internet hoặc gây phiền nhiễu. Giải pháp về thiết kế sản phẩm: giải thích lợi ích rõ ràng, yêu cầu chấp thuận phù hợp, cung cấp nút tắt/điều khiển thủ công, thông báo lỗi dễ hiểu và duy trì trải nghiệm khi ngoại tuyến nếu chức năng cho phép.

3.6.2 Dữ liệu và quyền riêng tư

Cần xác định mục đích thu thập, lượng dữ liệu tối thiểu, đối tượng có quyền truy cập, thời gian lưu và cách xóa dữ liệu. Dữ liệu vị trí, thói quen sinh hoạt hoặc thông tin sức khỏe có

thể nhạy cảm. Phân quyền kỹ thuật phải đi cùng quy trình tổ chức và yêu cầu pháp lý áp dụng; không nên thu dữ liệu chỉ vì cảm biến có khả năng đo.

3.6.3 Rủi ro trong vòng đời sản phẩm

Ngoài tấn công mạng còn có pin chai, mất hỗ trợ phần mềm, chuẩn truyền không tương thích, máy chủ ngừng vận hành, thay đổi nhà cung cấp và chi phí vận hành kéo dài. Một sản phẩm IoT phải có kế hoạch cập nhật firmware, sửa chữa, thay thế và ngừng hỗ trợ an toàn.

Theo bài giảng Chapter 1: An overview of IoT, trang PDF 47–49.

3.7 Đối chiếu cơ hội – yêu cầu thiết kế

Bảng 3.1: Chuyển cơ hội ứng dụng thành tiêu chí kỹ thuật

Giá trị kỳ vọng	Đầu vào dữ liệu cần thiết	Thách thức ưu tiên
Cảnh báo sự cố máy sớm	Rung, nhiệt, dòng điện, lịch sử vận hành	Độ tin cậy sensor, cảnh báo giả, bảo trì mạng
Giảm điện năng tòa nhà	Đo điện, hiện diện, giờ sử dụng	Quyền riêng tư, an toàn điều khiển, công suất nguồn
Theo dõi đội xe	Vị trí, thông số xe, lịch hành trình	Phủ sóng, bảo vệ dữ liệu vị trí, ngoại tuyến
Tưới tiêu theo tình trạng	Ẩm đất, thời tiết, lịch tưới	Hiệu chuẩn, pin và vỏ chống nước, trạng thái fail-safe
Quản lý kho lạnh	Nhiệt độ, cửa mở, mất điện	Lưu dữ liệu không gián đoạn, cảnh báo và dự phòng nguồn

3.8 Phân tích dự báo trong slide và giới hạn sử dụng

Trang PDF 46 chứa một infographic dự báo cho năm **2025** với một số chỉ tiêu về số kết nối, giá trị kinh tế và dung lượng dữ liệu. Vì đây là **dự báo tại thời điểm xây dựng slide** và bản gốc không cung cấp đầy đủ cách tính/định nghĩa các chỉ tiêu ngay trên trang, tài liệu này **không trình bày các con số đó như kết quả thực nghiệm đã được xác nhận**. Nội dung cần học là cơ sở suy luận: số nút gia tăng kéo theo nhu cầu lớn về quản lý thiết bị, cơ sở hạ tầng, bảo mật, năng lượng và phân tích dữ liệu.

Theo bài giảng Chapter 1: An overview of IoT, trang PDF 46.

3.9 Tổng kết: từ IoT tới giá trị thực tế

Bài giảng kết luận IoT được hình thành nhờ hội tụ công nghệ vô tuyến, vi cơ điện tử và Internet; hệ thống gồm cảm biến, thiết bị kết nối và tài nguyên lưu trữ lớn; công nghệ/giao thức mới liên tục thúc đẩy giá thành, tính khả dụng, hiệu quả năng lượng và bảo mật. Cần nhớ rằng **bài toán kỹ thuật chỉ có ý nghĩa khi gắn với yêu cầu sử dụng rõ ràng, chi phí vận hành và trách nhiệm đối với người dùng**.

Theo bài giảng Chapter 1: An overview of IoT, trang PDF 49.

Ghi nhớ

Tóm tắt thi: Nắm 10 lĩnh vực ứng dụng; biết phân tích hai scenario theo chuỗi dữ liệu–gateway–service–người dùng; thuộc bốn nhóm cơ hội/tác động trong slide và chín nhóm thách thức; giải thích được các cặp dễ nhầm: scalability/reliability, reliability/fault tolerance, data/information, encryption/access control.

3.10 Câu hỏi tự kiểm tra và bài tập tình huống

1. Liệt kê mười ứng dụng minh họa trong bài giảng. Chọn ba ứng dụng và nêu đại lượng cần đo.
2. Trong tình huống đồng hồ báo thức, đâu là dữ liệu đầu vào, đâu là quy tắc ra quyết định, đâu là thiết bị được điều khiển?
3. Trong tình huống ô tô kết nối, khác biệt giữa dữ liệu chẩn đoán tại xe và dịch vụ bảo dưỡng dự báo là gì?
4. Phân tích ba cơ hội: thị trường mới, thay đổi tạo giá trị, tăng khả năng quan sát thế giới vật lý.
5. Nêu năm thách thức kỹ thuật và đưa ra biện pháp giảm thiểu cho mỗi thách thức.
6. Vì sao tăng số nút có thể làm giảm độ tin cậy của hệ thống dù mỗi cảm biến đều hoạt động tốt?
7. Nếu thiết bị mất Internet, hệ thống nào vẫn cần xử lý tại chỗ? Hãy giải thích theo tiêu chí an toàn.
8. Thiết kế bằng lời một hệ theo dõi kho lạnh: ít nhất ba loại thành phần phần cứng/phần mềm, chu kỳ lấy mẫu, ngưỡng cảnh báo, dự phòng khi mất điện và cơ chế quyền truy cập.

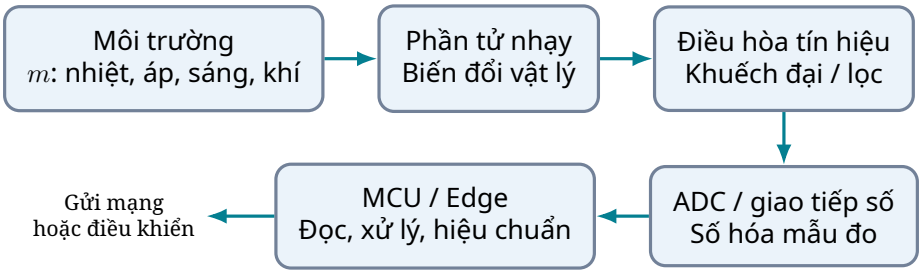
CHƯƠNG 4
Cảm biến (Sensors)

4.1 Mục tiêu học tập và vị trí trong kiến trúc IoT

Hoàn thành chương này, người học cần: định nghĩa được cảm biến; mô tả đường đi của đại lượng đo đến dữ liệu số; phân biệt cảm biến tích cực/thụ động và đầu ra tương tự/số; nhận diện các cảm biến thường gặp; tính toán giá trị ADC và lựa chọn giao tiếp phù hợp; thiết kế sơ bộ nút thu thập dữ liệu với chú ý về nguồn, sai số và hiệu chuẩn.

Theo slide bài giảng, kiến trúc IoT được hình dung qua ba tầng gần gũi: **Things / IoT Devices** thu thập dữ liệu vật lý; **Edge** xử lý gần nơi phát sinh dữ liệu; và **Cloud** lưu trữ, phân tích, quản lý thông qua kết nối mạng. Phần cứng nút cảm biến là cầu nối giữa thế giới vật lý và luồng thông tin số.

Theo Chapter 2: IoT Hardware, slide PDF 3–4, 16–17.



Hình 4.1: Chuỗi thu thập dữ liệu cảm biến, sơ đồ biên soạn mới.

4.2 Khái niệm và đặc tính chuyển đổi

Cảm biến (*sensor*) là phần tử hoặc thiết bị nhận một đại lượng vật lý, hóa học, sinh học và tạo ra tín hiệu đo có thể đọc, xử lý hoặc truyền đi. Đại lượng được đo ký hiệu m (*measurand*); tín hiệu đầu ra là s; quan hệ chuyển đổi được viết

$$\boxed{s = f(m).} \tag{4.1}$$

Trường hợp gần tuyến tính trong một miền làm việc: $s = km + b$. Hệ số $k = \Delta s/\Delta m$ được gọi là *độ nhạy* nếu các điều kiện đo phù hợp. Tín hiệu có thể là điện áp, dòng điện, điện trở, điện dung, tần số, xung hoặc chuỗi bit qua bus số. Trên thực tế, nhiều cảm biến cần *mạch kích thích* và mạch điều hòa để tạo ra đầu ra phù hợp với bộ xử lý.

Theo Chapter 2: IoT Hardware, slide PDF 4.

4.2.1 Các đại lượng ảnh hưởng chất lượng phép đo (mở rộng kỹ thuật)

Khoảng đo (range)
Khoảng đại lượng mà thiết bị cho phép đo theo công bố của nhà sản xuất; vượt khoảng đo có thể bão hòa hoặc hư hỏng.

Độ phân giải (resolution)
Thay đổi nhỏ nhất có thể phân biệt tại đầu ra; độ phân giải số không đồng nghĩa với độ chính xác.

Độ chính xác (accuracy)
Mức độ gần giá trị đo với giá trị chuẩn; thường được nêu theo $\pm$ đơn vị hoặc phần trăm.

Độ lặp lại (repeatability) và độ chính xác lặp (precision)
Mức phân tán kết quả trong điều kiện đo giống nhau; kết quả lặp tốt vẫn có thể lệch chuẩn.

Độ nhạy, độ tuyến tính, độ trễ
Mô tả mức đáp ứng, sai lệch so với mô hình tuyến tính và sự phụ thuộc vào chiều tăng/giảm đại lượng.

Thời gian đáp ứng, dải thông
Giới hạn tốc độ phản ứng, ảnh hưởng trực tiếp đến tần số lấy mẫu phù hợp.

Độ trôi (drift)
Biến đổi điểm không hoặc độ nhạy do nhiệt độ, lão hóa và môi trường. Phải xem xét hiệu chuẩn định kỳ.

4.3 Phân loại cảm biến

Slide cung cấp ba hướng phân loại: theo *nguồn năng lượng/tác động kích thích*, theo *đại lượng hay hiện tượng chuyển đổi* và theo *dạng đầu ra*. Không nên hiểu đây là ba cách phân loại loại trừ nhau: một cảm biến nhiệt điện trở có thể đồng thời được gọi là cảm biến nhiệt, cảm biến điện trở và cảm biến đầu ra analog khi ghép với mạch đọc.

Theo Chapter 2: IoT Hardware, slide PDF 7–12.

4.3.1 Cảm biến tích cực và thụ động

Bài giảng mô tả cảm biến **tích cực** (*active*) là cảm biến cần năng lượng kích thích bên ngoài, ví dụ radar/sonar chủ động phát sóng và đo phản xạ; cảm biến **thụ động** (*passive*) là cảm biến không cần chủ động phát tín hiệu đo, ví dụ cặp nhiệt điện tự phát sinh sức điện động từ chênh lệch nhiệt độ.

Ghi nhớ

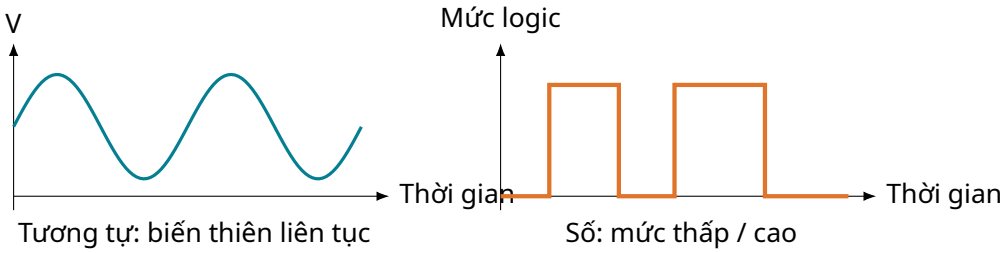
Phân biệt quan trọng: Trong tài liệu chuyên ngành, “active/passive” không phải lúc nào cũng được định nghĩa thống nhất. Việc cảm biến *không chủ động phát sóng* khác với việc *không cần nguồn cấp*. Camera thụ động thu ánh sáng môi trường nhưng vẫn cần điện để hoạt động. Một cảm biến áp suất kiểu điện trở/điện dung thường cần mạch kích thích, trong khi phần tử áp điện có thể tự tạo điện tích. Khi trả lời câu hỏi, nêu rõ tiêu chí phân loại được sử dụng.

Bảng 4.1: Đối chiếu nguyên lý theo cách mô tả của bài giảng

Tiêu chí	Tích cực (phát tín hiệu dò)	Thụ động (khai thác tín hiệu sẵn có)
Nguyên lý	Phát siêu âm, vô tuyến hoặc ánh sáng rồi đo đáp ứng	Thu tín hiệu từ đại lượng tự nhiên hoặc chuyển đổi trực tiếp
Ví dụ	Radar đo khoảng cách; siêu âm HC-SR04	Cặp nhiệt điện; quang trở nhận ánh sáng
Ưu điểm	Chủ động phép đo, ít phụ thuộc một số nguồn kích thích bên ngoài	Có thể đơn giản, ít ảnh hưởng tới đối tượng
Lưu ý	Công suất phát, phản xạ và nhiễu đồng kênh	Không đồng nghĩa toàn bộ thiết bị không dùng điện

4.3.2 Tín hiệu tương tự và tín hiệu số

Tín hiệu tương tự (*analog*) thay đổi liên tục theo thời gian/giá trị (trong mô hình lý tưởng). **Tín hiệu số** (*digital*) biểu diễn thông tin bằng các mức rời rạc; tín hiệu logic vật lý thường biểu diễn mức LOW/HIGH (0/1), còn dữ liệu đo số có thể mã hóa hàng chục bit và nhiều giá trị, không bị giới hạn ở hai giá trị của đại lượng đo.



Hình 4.2: Hai dạng tín hiệu thường gặp ở đầu vào vi điều khiển.

Một *cảm biến analog* có thể xuất điện áp tỷ lệ đại lượng đo (hoặc thay đổi điện trở cần mạch phân áp). Bộ điều khiển cần ADC để chuyển thành mã số. Một *cảm biến số* tích hợp ADC, xử lý/hiệu chuẩn hoặc bộ giao tiếp số, truyền kết quả qua I^2C, SPI, UART, 1-Wire; cũng có thiết bị chỉ tạo xung logic. Do đó nhận định trong slide rằng cảm biến số "có dải đo thấp" không phải tính chất phổ quát: phải đối chiếu thông số của *từng model*.

Theo Chapter 2: IoT Hardware, slide PDF 9–12.

4.3.3 Phân loại theo đại lượng và cơ chế

Ví dụ cảm biến trong slide gồm: nhiệt độ, ẩm đất, độ ẩm không khí, hồng ngoại, khí gas, CO_2, nút nhấn, alcohol, siêu âm, LDR, rain, photodiode, proximity, PIR, gyroscope. Cần phân biệt *nguyên lý* và *board/module*: cùng là cảm biến nhiệt độ nhưng có module analog, digital và cảm biến tiếp xúc/không tiếp xúc khác nhau.

Theo Chapter 2: IoT Hardware, slide PDF 5–6, 11–12.

4.4 Kết nối cảm biến với vi điều khiển (mở rộng kỹ thuật)

Bảng 4.2: Các cảm biến minh họa trong bài giảng và công dụng

Nhóm	Ví dụ trong slide	Đại lượng / ứng dụng
Nhiệt độ	Temperature sensor; cặp nhiệt điện	Giám sát nhiệt độ, bảo vệ thiết bị
Ẩm	Humidity, soil moisture	Độ ẩm không khí hoặc độ ẩm đất; không dùng lẫn hai đơn vị đo
Quang	LDR, photodiode, IR optical	Cường độ ánh sáng, phát hiện vật/đường line
Khoảng cách	Ultrasonic, IR proximity	Khoảng cách, nhận diện chướng ngại
Chuyển động	Accelerometer, gyroscope, PIR	Gia tốc, tốc độ góc, phát hiện chuyển động hồng ngoại
Môi trường	Gas, CO_2, rain, alcohol	Chỉ thị khí hoặc lượng mưa; phân biệt cảm biến khí tổng hợp và CO_2 chuyên dụng
Tương tác	Digital push button	Trạng thái nhấn/thả, thường là tín hiệu logic
Áp suất	Pressure sensor	Áp suất khí/quá trình, suy ra cao độ trong điều kiện phù hợp

4.4.1 ADC, lượng tử hóa và ví dụ tính toán

ADC N bit tạo ra 2^N mức mã, thông thường từ 0 đến $2^N - 1$. Với ADC đơn cực lý tưởng tham chiếu V_{ref} và điện áp hợp lệ trong $[0, V_{\text{ref}}]$, công thức quy đổi xấp xỉ từ mã D là

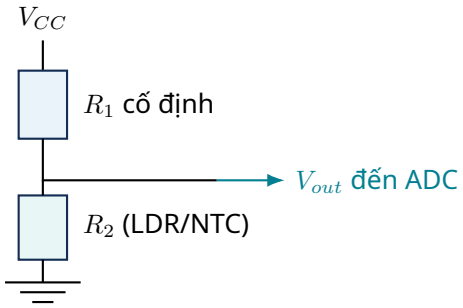
$$\boxed{V_{\text{in}} \approx \frac{D}{2^N - 1} V_{\text{ref}}} \qquad (0 \le D \le 2^N - 1). \tag{4.2}$$

Bước lượng tử hóa danh định thường viết $\Delta V \approx V_{\text{ref}}/2^N$; một số cách quy đổi cuối thang dùng $2^N - 1$. Phải tuân theo quy ước của ADC và thư viện cụ thể. Ví dụ ADC 10 bit, $V_{\text{ref}} = 5\,\text{V}$, nhận $D = 512$: $V_{\text{in}} \approx 512 \times 5/1023 \approx 2.50\,\text{V}$. Đây là **độ phân giải mã**, không nói trước sai số hệ thống.

4.4.2 Cảm biến điện trở: mạch phân áp

Một phần tử cảm biến thay đổi điện trở, chẳng hạn LDR hay NTC, cần được đo dưới dạng điện áp. Với R_1 nối từ nguồn V_{CC} đến nút đo, R_2 nối từ nút đo xuống GND:

$$V_{\text{out}} = V_{CC} \frac{R_2}{R_1 + R_2}. \tag{4.3}$$



Hình 4.3: Đấu nối phân áp nguyên lý. Kiểm tra điện áp nguồn trước khi đưa vào ADC.

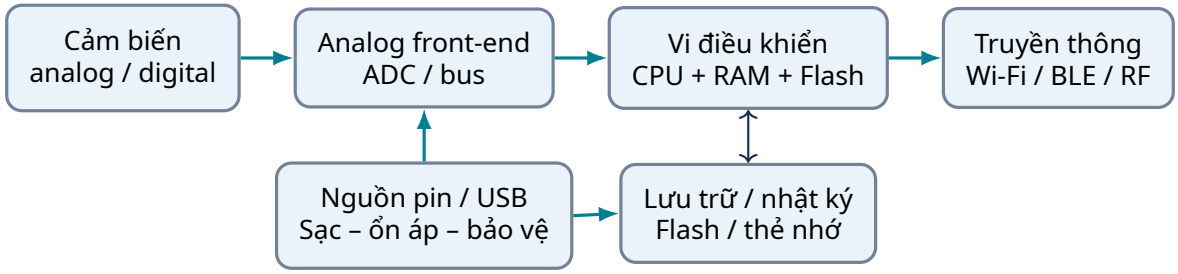
4.4.3 I²C, SPI, UART và ngõ vào số

Bảng 4.3: Khi nào dùng giao tiếp nào?

Giao tiếp	Dây và nguyên lý	Lưu ý thiết kế
GPIO số	Một chân đọc HIGH/LOW, ngắt cạnh	Cần điện trở kéo lên/kéo xuống phù hợp; chống dội nút nhấn
ADC	Một chân analog tham chiếu mức điện áp	Chống aliasing, lọc nhiễu, hiệu chuẩn, không quá áp
I^2C	SDA/SCL, địa chỉ thiết bị trên bus	Điện trở kéo lên, địa chỉ trùng, điện áp logic, chiều dài bus
SPI	SCLK, MOSI, MISO, CS	Nhanh, nhiều dây; mỗi slave thường cần CS
UART	TX/RX, mức baud, định dạng bit	Nối chéo TX-RX, GND chung, không nhầm TTL với RS-232
1-Wire	Dây dữ liệu cùng GND, giao thức định thời	Điện trở kéo lên; thời gian và chiều dài đường truyền

4.5 Mô hình nút cảm biến IoT

Theo sơ đồ trang 16–17 slide, nút IoT gồm *Sensors* + *Board* + *Connectivity* và được phân rã thành cảm biến, mạch tiền xử lý/ADC, vi điều khiển, bộ nhớ, truyền thông có dây/không dây và khối nguồn. Lớp ứng dụng còn cần firmware, định danh và bảo mật.



Hình 4.4: Sơ đồ khối cấu trúc nút IoT tái dựng từ nội dung bài giảng.

Theo Chapter 2: IoT Hardware, slide PDF 16–17.

4.5.1 Nguồn và độ tin cậy đo

Thiết kế nút cảm biến cần lưu ý dòng khởi động và dòng đỉnh của RF, mức nghỉ của cảm biến, chu kỳ đo, khả năng giảm công suất khi ngủ và tính toàn vẹn dữ liệu. Phương pháp ước lượng cơ bản:

$$I_{\text{avg}} = \sum_i D_i I_i, \qquad T_{\text{h}} \approx \frac{C_{\text{usable}}\,(\text{mAh})}{I_{\text{avg}}\,(\text{mA})}. \tag{4.4}$$

Đây là ước tính lý tưởng theo dung lượng *khả dụng*, chưa tính hiệu suất DC/DC, tự xả, suy giảm pin và nhiệt độ. Chu kỳ lấy mẫu nên dựa trên dải tần tín hiệu, yêu cầu trễ, tuổi thọ pin và chi phí truyền thông; lấy mẫu nhanh hơn không mặc nhiên làm dữ liệu chính xác hơn.

4.6 Ví dụ thiết kế: đo nhiệt độ và điều khiển quạt

Một cảm biến đọc nhiệt độ theo chu kỳ, MCU lọc trung bình trượt, so sánh với hai ngưỡng và xuất tín hiệu đến mạch công suất điều khiển quạt. Hai ngưỡng $T_{\text{on}} > T_{\text{off}}$ tạo *hysteresis*, tránh đóng ngắt liên tục. Nếu sensor báo lỗi, hệ thống cần chính sách rõ ràng: tắt an toàn, báo lỗi hoặc bật quạt phòng ngừa tùy nguy cơ.

Ví dụ giả định: bật quạt khi $T \geq 32°$C, tắt khi $T \leq 29°$C. Mức này chỉ minh họa thuật toán, không phải nhiệt độ khuyến nghị cho một loại thiết bị cụ thể. MCU không được trực tiếp cấp dòng cho mô-tơ.

4.7 Câu hỏi ôn tập và bài tập

1. Trình bày $s = f(m)$, phân biệt tín hiệu đầu ra của cảm biến với đại lượng vật lý cần đo. Nêu một ví dụ có đường truyền xử lý hoàn chỉnh.
2. Theo phân loại của bài giảng, giải thích tại sao radar và sonar được coi là chủ động, cặp nhiệt điện là thụ động; nhận xét việc dùng từ "không cần năng lượng".
3. So sánh cảm biến analog với cảm biến digital; giải thích vì sao một cảm biến có giao tiếp I^2C có thể xuất nhiều giá trị đo dù tín hiệu trên dây là logic 0/1.
4. Tính mã ADC 10 bit gần đúng khi điện áp vào 1.65 V và $V_{ref} = 3.3$ V. *Đáp số:* gần 512.
5. Với phân áp $R_1 = 10\,\text{k}\Omega$, $R_2 = 5\,\text{k}\Omega$ và $V_{CC} = 3.3$ V, tính V_{out}. *Đáp số:* 1.1 V.
6. Vẽ lại nút IoT dùng cảm biến nhiệt độ số, ESP32, Wi-Fi, nguồn pin và cảnh báo LED; ghi bus, điện áp logic, hướng truyền dữ liệu.
7. Phân tích ảnh hưởng của nhiệt độ, nhiễu nguồn, vị trí lắp đặt, sai số hiệu chuẩn đến độ tin cậy phép đo.

Ghi nhớ

Tóm tắt cần nhớ: Cảm biến biến đổi đại lượng thực thành tín hiệu; hiểu đúng đơn vị, dải đo, sai số và điện áp đầu ra quan trọng hơn việc ghi nhớ tên module. Ba trục phân loại (kích thích, hiện tượng, dạng đầu ra) có thể chồng lấp. Một nút IoT hoàn chỉnh cần cảm biến, mạch ghép nối, xử lý, nguồn và truyền thông.

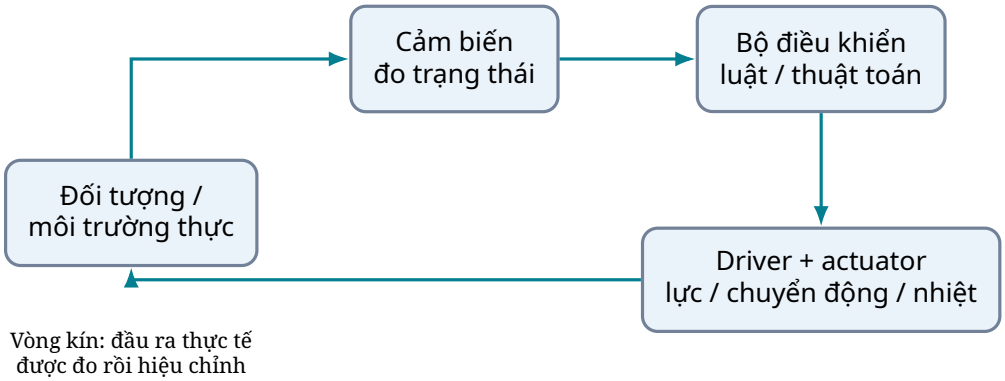
CHƯƠNG 5
Cơ cấu chấp hành (Actuators)

5.1 Mục tiêu và khái niệm

Sau chương này, người học cần mô tả được vai trò cơ cấu chấp hành trong IoT; phân biệt cảm biến/đầu vào với đầu ra vật lý; chọn sơ bộ phần tử chấp hành; hiểu nguyên lý GPIO, PWM và mạch driver; biết các ràng buộc nguồn, bảo vệ và an toàn khi ghép MCU với tải.

Theo slide, **cơ cấu chấp hành** (*actuator*) là thiết bị nhận tín hiệu điều khiển (thường là điện) và tạo ra chuyển động cơ học hoặc một thay đổi vật lý như ánh sáng, âm thanh, nhiệt độ, áp lực, trạng thái đóng mở. Các ví dụ minh họa bao gồm động cơ điện, động cơ bước, LED, LCD, ma trận LED, còi/buzzer. Trong ngôn ngữ hẹp, "actuator" có thể chỉ cơ cấu tạo chuyển động; ở phạm vi bài giảng IoT, từ này được dùng rộng hơn để chỉ *thiết bị đầu ra tác động lên môi trường hoặc người sử dụng*.

Theo Chapter 2: IoT Hardware, slide PDF 13–15.



Hình 5.1: Vòng điều khiển phản hồi: cảm biến là đường đo; actuator là đường tác động.

5.2 So sánh cảm biến và cơ cấu chấp hành

5.3 Nhóm phần tử chấp hành thường gặp

5.3.1 LED, màn hình và còi báo

LED biến điện năng thành ánh sáng, phù hợp báo trạng thái. LED đơn cần điện trở hạn dòng $R \approx (V_{CC} - V_F)/I$ khi điện áp cấp lớn hơn điện áp thuận V_F và phù hợp với khả năng dòng chân MCU. Chẳng hạn với $V_{CC} = 3.3$ V, $V_F = 2.0$ V, mục tiêu $I = 5$ mA: $R \approx 260\,\Omega$, có thể chọn điện trở chuẩn gần đó và tính lại dòng. Màn hình LCD, ma trận LED cần nhiều tín hiệu dữ

Bảng 5.1: Hai vai trò trong hệ IoT, tổng hợp từ slide 14

Thuộc tính	Cảm biến (sensor)	Chấp hành (actuator)
Vai trò	Đọc hoặc cảm nhận thế giới thực	Tác động vào thế giới thực
Hướng dữ liệu	Vật lý → tín hiệu → MCU	MCU → lệnh / driver → thay đổi vật lý
Vị trí chức năng	Nhánh đầu vào hệ điều khiển	Nhánh đầu ra hệ điều khiển
Ví dụ	Nhiệt kế, micro, IMU, từ kế	Quạt, LED, loa, van điện từ, servo
Sai lầm hay gặp	Nhầm mức logic sensor là tải	Nối động cơ trực tiếp với chân GPIO

liệu và mạch driver; buzzer có loại *active* tự tạo âm khi cấp điện và *passive* cần tín hiệu dao động. Tên "active/passive" của buzzer không đồng nghĩa cách phân loại cảm biến.

5.3.2 Động cơ DC, động cơ bước và servo

Động cơ DC: tốc độ xấp xỉ điều khiển qua điện áp hiệu dụng/PWM; chiều quay có thể đảo bằng cầu H. Dòng khởi động/khóa trục có thể lớn hơn nhiều so với dòng chạy không tải, do đó cần driver và nguồn được tính theo tải thực.

Động cơ bước (stepper): quay theo từng bước rời rạc khi driver được cấp chuỗi xung pha. Với động cơ có góc bước θ_{step}, nếu điều khiển bước đầy đủ và không bị mất bước: $N_{rev} = 360^\circ/\theta_{step}$. Ví dụ động cơ 1.8°/bước cần 200 bước/vòng. Vi bước (microstepping) tăng số vi bước lệnh trên một vòng nhưng không bảo đảm độ chính xác cơ khí tương ứng. Slide sử dụng stepper để minh họa điều khiển vị trí trong cánh tay robot.

Servo RC: thường có bộ điều khiển bên trong, nhận xung điều khiển theo chu kỳ; góc, dải xung và tốc độ tùy servo. Cần nguồn đủ dòng, GND chung và không mặc định rằng mọi servo hỗ trợ cùng phạm vi góc.

Theo Chapter 2: IoT Hardware, slide PDF 13.

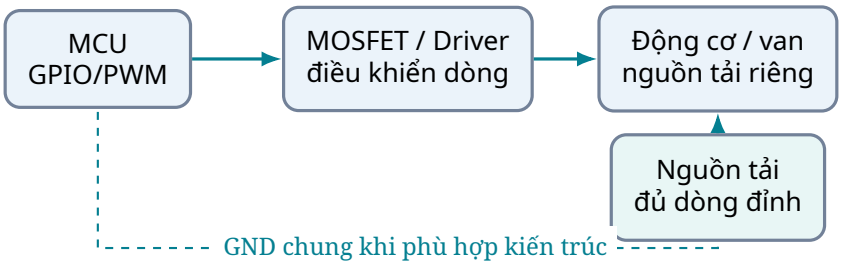
5.3.3 Rơ-le, van điện từ, tải nhiệt và cơ cấu tuyến tính

Rơ-le dùng cuộn hút để thay đổi trạng thái tiếp điểm và cách ly chức năng điều khiển khỏi tải, nhưng phải đáp ứng định mức điện áp/dòng, khoảng cách điện và chuẩn an toàn. Van điện từ dùng từ trường điều khiển dòng lưu chất; động cơ tuyến tính/solenoid dùng lực điện từ để kéo hoặc đẩy. Dây đốt nóng và module Peltier biến điện thành nhiệt/lạnh tương đối; cần cảm biến phản hồi, bảo vệ quá nhiệt và nguồn công suất phù hợp.

5.4 Nguyên lý ghép mạch điều khiển công suất

5.4.1 Vì sao GPIO không cấp trực tiếp được tải lớn?

GPIO xuất mức logic và dòng giới hạn bởi datasheet MCU; mô-tơ, van, rơ-le tiêu thụ dòng lớn và có thể phát xung quá áp khi ngắt. Mạch driver (MOSFET, BJT, cầu H, IC chuyên dụng) chuyển tín hiệu mức thấp thành chuyển mạch tải. Với tải cuộn cảm DC, diode *flyback* mắc ngược chiều dẫn bình thường tạo đường cho dòng cảm khi ngắt. Tải AC/công suất lớn cần mạch và biện pháp an toàn riêng, không sử dụng sơ đồ DC đơn giản một cách máy móc.



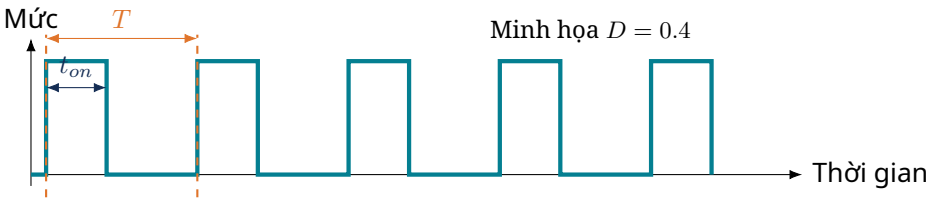
Hình 5.2: Phân tách tín hiệu điều khiển và đường công suất.

5.4.2 PWM: điều chỉnh công suất trung bình

PWM (*pulse-width modulation*) đóng/ngắt tải theo chu kỳ T với thời gian ON là t_{on}; hệ số lấp đầy:

$$\boxed{D = \frac{t_{on}}{T}}, \qquad 0 \le D \le 1, \qquad f_{PWM} = \frac{1}{T}. \tag{5.1}$$

Với tải có tính quán tính/lọc đủ lớn, công suất hoặc điện áp *trung bình* có thể phụ thuộc D. Ví dụ PWM mức cao 3.3 V, $D = 0.40$ cho điện áp trung bình lý tưởng khoảng 1.32 V *tại tín hiệu PWM*, nhưng không nên coi đó là điện áp DC sạch và không suy ra ngay mô-men động cơ. Tần số PWM chọn theo driver, tiếng ồn, hiệu suất và yêu cầu điều khiển.



Hình 5.3: Xung PWM 40% duty cycle (sơ đồ lý tưởng).

5.5 Đặc tính kỹ thuật khi chọn actuator

Bảng 5.2: Bảng kiểm tham số thiết kế

Tham số	Câu hỏi cần trả lời	Rủi ro nếu bỏ qua
Công suất / dòng	Điện áp danh định? Dòng chạy, dòng đỉnh, dòng khởi động?	Sụt áp, reset MCU, quá nhiệt
Driver	Cần MOSFET, relay, cầu H, IC điều khiển bước?	Hỏng GPIO, không đảo chiều được
Tốc độ / vị trí	Tốc độ, hành trình, mô-men và độ chính xác cần bao nhiêu?	Mất bước, quá tải cơ khí
Điều khiển	On/off, PWM, xung bước, UART, I^{2}C?	Không tương thích MCU/driver
Phản hồi	Có encoder, công tắc hành trình, cảm biến dòng?	Không phát hiện kẹt tải
Điện từ / EMC	Tải cảm có diode dập xung, tụ, snubber phù hợp?	Reset, nhiễu sensor/RF
An toàn	Khi lỗi mất mạng hoặc MCU treo, tải về trạng thái nào?	Tải hoạt động không kiểm soát

Tham số	Câu hỏi cần trả lời	Rủi ro nếu bỏ qua
Cơ khí	Giới hạn nhiệt, lắp gá, rung, chống nước?	Mòn, hỏng, mất ổn định

5.6 Điều khiển vòng hở và vòng kín

Vòng hở (*open-loop*) phát lệnh mà không đo lại kết quả. Ví dụ cấp 100 xung bước và giả định trục quay đúng 100 bước; mô hình đơn giản, chi phí thấp nhưng có thể sai lệch do kẹt tải. **Vòng kín** (*closed-loop*) thu phản hồi để tính sai lệch $e(t) = r(t) - y(t)$, sau đó điều chỉnh lệnh $u(t)$. Các thuật toán thường gặp gồm on/off có hysteresis, PID và điều khiển trạng thái.

Ví dụ nhà thông minh: đặt nhiệt độ mục tiêu, sensor đo không khí, MCU xử lý, relay/MOSFET điều khiển quạt. Với cơ cấu chấp hành động cơ, nếu cần bảo đảm tốc độ thực cần encoder hoặc cảm biến phản hồi thích hợp.

5.7 Ví dụ chương trình Arduino: LED bằng PWM

Đoạn mã sau minh họa sử dụng bo Arduino Uno cổ điển với một chân PWM phù hợp. Chọn điện trở hạn dòng LED theo phần cứng thực tế; `analogWrite()` tạo PWM ở nhiều bo Arduino, *không* phải DAC analog thật.

Listing 5.1: Tăng/giảm độ sáng LED theo dạng tam giác

```
const int LED_PIN = 9;
void setup() {
  pinMode(LED_PIN, OUTPUT);
}
void loop() {
  for (int duty = 0; duty <= 255; duty += 5) {
    analogWrite(LED_PIN, duty);
    delay(25);
  }
  for (int duty = 255; duty >= 0; duty -= 5) {
    analogWrite(LED_PIN, duty);
    delay(25);
  }
}
```

Mã chỉ điều khiển LED công suất nhỏ trong giới hạn cho phép; muốn điều khiển bơm/van/động cơ phải thêm driver, cấp nguồn và bảo vệ.

5.8 Bài tập và câu hỏi ôn tập

1. Nêu ít nhất bốn khác biệt giữa cảm biến và cơ cấu chấp hành theo luồng năng lượng, thông tin và chức năng.
2. Tại sao động cơ bước điều khiển vị trí chính xác hơn DC motor vòng hở trong nhiều tình huống, và khi nào vẫn cần encoder?
3. Một động cơ có góc bước $1.8°$; muốn quay $90°$ ở chế độ bước đầy đủ, cần bao nhiêu bước? *Đáp số:* 50 bước.

4. PWM có $T = 10$ ms và thời gian ON 3.5 ms. Tính duty, tần số. *Đáp số:* 35%, 100 Hz.
5. Vẽ mạch khối ESP32 $\rightarrow$ driver MOSFET $\rightarrow$ bơm DC, xác định nguồn tải, GND, bảo vệ diode và tín hiệu điều khiển.
6. Nêu cách xử lý nếu sensor mất kết nối khi actuator đang bật. Vì sao không có chính sách lỗi chung cho mọi thiết bị?

Ghi nhớ

Tóm tắt cần nhớ: Actuator tạo tác động vật lý từ tín hiệu điều khiển. GPIO và driver là hai tầng khác nhau. Cần chọn nguồn theo dòng đỉnh, thiết kế bảo vệ tải cảm, phản hồi và trạng thái an toàn. PWM là điều chế độ rộng xung; không phải điện áp analog thuần.

CHƯƠNG 6

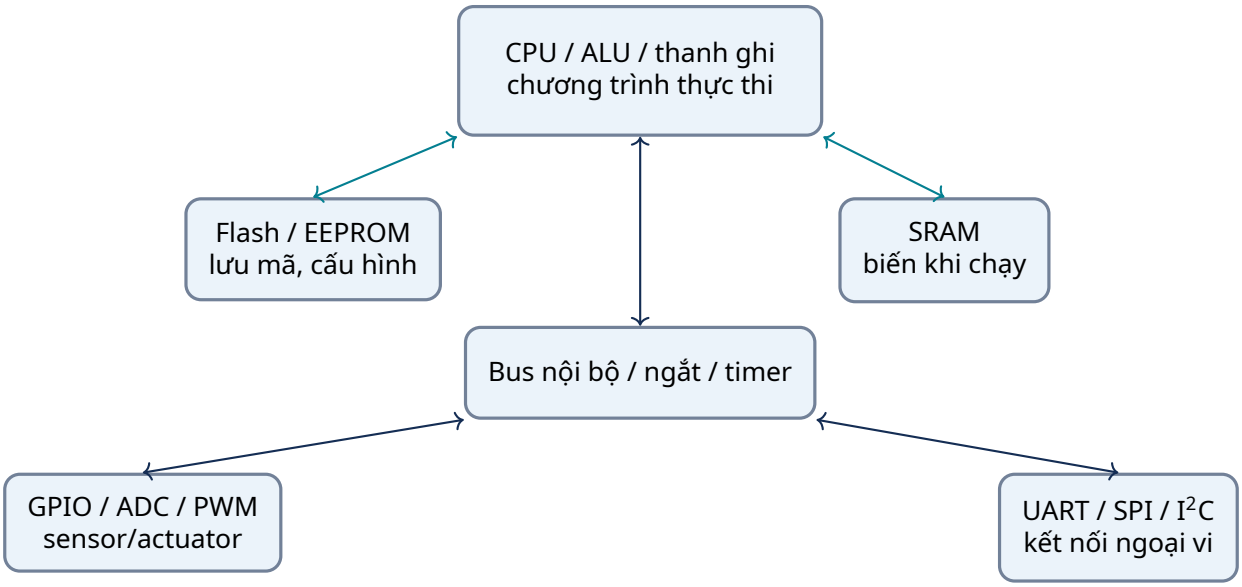
Vi điều khiển và nền tảng phần cứng IoT

6.1 Kiến trúc hệ thống nhúng và mục tiêu học tập

Sau chương này, người học cần hiểu sự khác nhau giữa chip vi điều khiển, bo phát triển và máy tính đơn bo; biết cấu trúc CPU–Flash–RAM–I/O; mô tả Arduino Uno, Leonardo, Mega, Nano, ATmega328P, ESP8266, ESP32, Intel Edison, Raspberry Pi; hiểu thao tác lập trình Arduino IDE, nối cảm biến, dùng UART và lựa chọn xử lý ở Edge hay Cloud.

Vi điều khiển (*microcontroller/MCU*) là vi mạch tích hợp bộ xử lý, bộ nhớ và ngoại vi vào một hệ lập trình được, tối ưu cho các tác vụ nhúng. Một **bo phát triển** (*development board*) đưa chip ra các chân tiện dụng và có thêm mạch nguồn, USB, clock, bộ nạp. Một **máy tính đơn bo** (*single-board computer/SBC*) như Raspberry Pi thường có SoC mạnh hơn, RAM ngoài, lưu trữ và khả năng chạy hệ điều hành đa nhiệm. Do đó **Arduino không đồng nghĩa với chip ATmega**, và **Raspberry Pi không nên được coi là MCU theo nghĩa hẹp**.

Theo Chapter 2: IoT Hardware, slide PDF 18, 22–26, 73–78.



Hình 6.1: Kiến trúc MCU mức khái niệm (sơ đồ mới, không phải sơ đồ silicon cụ thể).

6.2 ATmega328P: nền tảng AVR 8 bit

Slide trình bày ATmega328P là MCU AVR 8-bit hiệu năng tốt, công suất thấp, bộ nhớ Flash 32 KB, SRAM 2 KB, EEPROM 1 KB, tối đa 23 đường I/O lập trình được trên chip, dải điện áp hoạt động tùy tần số/phiên bản 1.8–5.5 V, vùng nhiệt độ công bố trong slide -40 đến $85\,^{\circ}$C. Các giá trị dòng tiết kiệm năng lượng đưa ra trong slide (active 0.2 mA, power-down $0.1\,\mu$A, power-save $0.75\,\mu$A) *chỉ là số liệu dưới điều kiện kiểm thử riêng*, không thể mặc nhiên dùng cho bo Arduino Uno khi có LED, bộ ổn áp và USB-UART.

Theo Chapter 2: IoT Hardware, slide PDF 19–21.

6.2.1 Flash, SRAM, EEPROM: ba vai trò khác nhau

Bảng 6.1: Bộ nhớ của ATmega328P theo bài giảng

Loại	Dung lượng	Chức năng
Flash	32 KB	Chứa chương trình và dữ liệu tĩnh, giữ khi mất nguồn
SRAM	2 KB	Biến lúc chạy, stack/heap; mất khi tắt nguồn
EEPROM	1 KB	Lưu cấu hình nhỏ khi mất nguồn; có giới hạn số chu kỳ ghi

Nếu khai báo quá nhiều biến động (mảng lớn, chuỗi, bộ đệm), hệ thống có thể lỗi do SRAM cạn dù Flash còn chỗ. Với dữ liệu ghi lặp nhiều, phải quan tâm mài mòn EEPROM và chiến lược ghi tối thiểu.

6.2.2 Chân nguồn, I/O và nạp chương trình

ATmega328P ở gói PDIP 28 chân thường có VCC, AVCC, GND, AREF, RESET, XTAL1/XTAL2 và các nhóm PORTB/C/D. Một chân vật lý có thể kiêm nhiều chức năng (GPIO, ADC, UART, SPI, PWM, ngắt), do đó *số chân package* không phải số GPIO thực dùng độc lập. Trong slide, giao tiếp nạp chương trình minh họa các tín hiệu RESET, SCK, MOSI, MISO, VCC, GND theo chuẩn ISP. Bootloader Arduino giúp nạp sketch qua USB-to-serial trên bo, trong khi ISP cho phép lập trình trực tiếp khi cần.

Theo Chapter 2: IoT Hardware, slide PDF 19, 21, 25, 27.

6.3 Arduino: hệ sinh thái phần cứng/phần mềm mở

Arduino là nền tảng điện tử mã nguồn mở gắn với bo và công cụ lập trình dễ tiếp cận, hỗ trợ nhiều hệ điều hành. Slide nhấn mạnh chi phí thấp, hỗ trợ đa nền tảng, môi trường lập trình đơn giản, phần mềm và phần cứng mở. Nội dung lịch sử trong slide nhắc dự án xuất phát từ môi trường giáo dục thiết kế tương tác tại Ivrea (Ý), phát triển thành hệ sinh thái bo, thư viện và shield rộng lớn.

Theo Chapter 2: IoT Hardware, slide PDF 22–24.

6.3.1 Các bo Arduino trong bài giảng

Slide còn nhắc EEPROM 1 KB trên Uno/Nano/Leonardo, 4 KB trên Mega; dòng chân I/O và dải nguồn phải xét theo MCU, điện áp cấp *cho chip* khác với điện áp cho chân VIN hoặc jack bo mạch. Một số số lượng digital pin trong slide bao gồm chân đa chức năng, do vậy cần xem

Bảng 6.2: Các bo điển hình theo slide; số liệu áp dụng cho phiên bản cổ điển được nêu

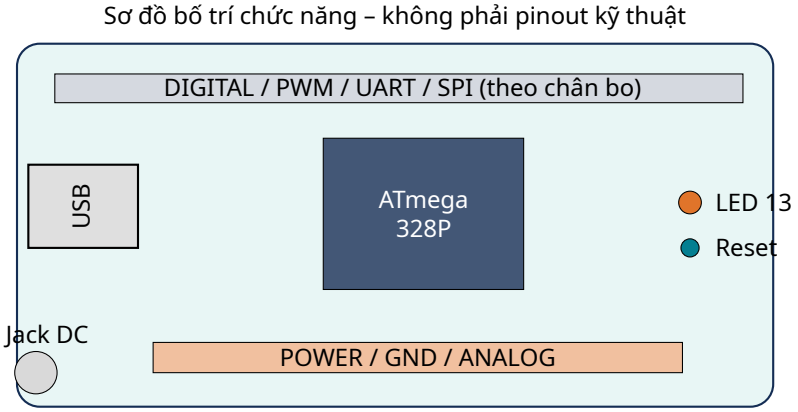
Bo	MCU	Flash / SRAM	Đầu vào/ra và ghi chú
Uno Rev3 (cổ điển)	ATmega328P, 8-bit	32 KB / 2 KB	14 chân digital, 6 analog; USB loại B; bootloader
Leonardo	ATmega32U4, 8-bit	32 KB / 2.5 KB	Tài liệu slide nêu 23 digital, 12 analog; USB tích hợp MCU
Mega 2560	ATmega2560, 8-bit	256 KB / 8 KB	54 digital, 16 analog; phù hợp khi cần nhiều chân
Nano (ATmega328)	ATmega328, 8-bit	32 KB / 2 KB	Thiết kế nhỏ; slide nêu 8 ngõ analog, 22 digital tổng hợp

sơ đồ chân của *phiên bản bo* cụ thể. Arduino Uno mới (ví dụ R4) không dùng ATmega328P; không thể suy thông số từ bảng này cho mọi sản phẩm tên "Uno".

Theo Chapter 2: IoT Hardware, slide PDF 25–30.

6.3.2 Phân tích một bo Uno cổ điển

Sơ đồ trang 26 slide chỉ ra các khu vực: hàng chân Digital, LED chân 13, LED báo nguồn, ATmega chính, hàng Analog, chân GND/5V, jack nguồn, đèn TX/RX, cổng USB và nút Reset. Chân Arduino D0/D1 thường gắn với UART RX/TX, nên dùng đồng thời với module UART và Serial Monitor có thể gây tranh chấp.



Hình 6.2: Sơ đồ khối bố trí thành phần tiêu biểu của Arduino Uno đời ATmega328P.

6.3.3 Shield, breadboard, kit và các nền tảng thay thế

Shield (*bo mở rộng*) cắm vào một số bo Arduino qua hàng chân tiêu chuẩn để bổ sung Ethernet, relay, màn hình, driver mô-tơ hoặc kết nối RF. Cần kiểm tra xung đột chân và tương thích mức logic. **Breadboard** (*bảng cắm thử không hàn*) thích hợp thử nghiệm nguyên mẫu, nhưng không lý tưởng cho dòng lớn, điện áp cao, tần số RF và rung động. Arduino kit thường gom các linh kiện, dây, sensor, servo và tài liệu thực hành.

Các phương án được slide giới thiệu gồm LaunchPad MSP430, Particle Photon, NodeMCU ESP8266, SparkFun Thing Plus, Feather HUZZAH, PocketBeagle, Seeeduino Nano, RedBoard Artemis và STM32F3 Discovery. Mỗi nền tảng có hệ sinh thái, CPU, mức logic và công cụ nạp khác nhau; được liệt kê để so sánh *khả năng lựa chọn*, không phải toàn bộ tương thích chân và mã Arduino.

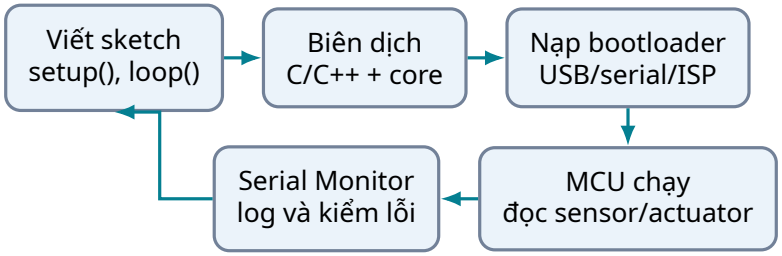
Theo Chapter 2: IoT Hardware, slide PDF 31–37.

6.4 Lập trình Arduino IDE và ngôn ngữ Sketch

6.4.1 Chu trình công cụ và chương trình

Slide gọi chương trình tải lên Arduino là **sketch** (*bản phác chương trình*); ngôn ngữ thực tế dựa trên C/C++ và các hàm của Arduino core. IDE cho phép lựa chọn board, cổng serial, quản lý thư viện, biên dịch, nạp và quan sát Serial Monitor. Quá trình cài board bên thứ ba (như ESP8266) thường đi qua Boards Manager; thư viện cài qua Library Manager hoặc các hình thức khác tùy dự án.

Theo Chapter 2: IoT Hardware, slide PDF 38–41, 51–52.



Hình 6.3: Chu trình thiết kế – nạp – kiểm thử firmware Arduino.

6.4.2 Hai hàm `setup()` và `loop()`

`setup()` chạy một lần sau khi khởi động hoặc reset để cấu hình chân, serial, thư viện và trạng thái ban đầu. `loop()` chạy lặp lại; muốn hệ thống phản ứng nhanh với nhiều công việc, nên ưu tiên lập lịch theo `millis()` thay vì nhiều lệnh `delay()` dài. Khi có lỗi hoặc yêu cầu dừng vĩnh viễn cần xử lý có chủ đích, tránh khối vòng lặp vô hạn vô tình.

Theo Chapter 2: IoT Hardware, slide PDF 41–51.

Listing 6.1: Cấu trúc sketch tối thiểu

```
const int LED_PIN = LED_BUILTIN;
void setup() {
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(9600);
}
void loop() {
  digitalWrite(LED_PIN, HIGH);
  delay(500);
  digitalWrite(LED_PIN, LOW);
  delay(500);
}
```

6.4.3 Lệnh và khái niệm quan trọng

Bảng 6.3: Tóm tắt cú pháp được giới thiệu trong slide 40–50

Lệnh / cấu trúc	Ý nghĩa và chú ý
`//`, `/*...*/`	Chú thích một dòng và nhiều dòng; trình biên dịch bỏ qua nội dung chú thích.
`#define`, `#include`	Định nghĩa macro và thêm phần khai báo thư viện/header; khác với khai báo biến.

Lệnh / cấu trúc	Ý nghĩa và chú ý
`if/else`	Rẽ nhánh nếu điều kiện đúng/sai; điều kiện là biểu thức logic.
`for`, `while`	Lặp có đếm hoặc lặp khi điều kiện còn đúng; đề phòng vòng lặp vô hạn.
`pinMode(pin,mode)`	Đặt chế độ chân (`INPUT`, `OUTPUT`, `INPUT_PULLUP` khi hỗ trợ).
`digitalRead(pin)`	Đọc trạng thái logic chân số (HIGH/LOW).
`digitalWrite(pin,value)`	Ghi mức HIGH/LOW lên GPIO output.
`analogRead(pin)`	Đọc ADC trả mã số; độ rộng/mức tham chiếu tùy bo.
`analogWrite(pin,value)`	Trên Uno cổ điển tạo PWM ở các chân hỗ trợ; độ rộng giá trị mặc định thường 0–255.
`delay(ms)`	Dừng tác vụ một khoảng thời gian (không nên dùng tràn lan trong tác vụ thời gian thực).
`abs()`, `max()`, `min()`, `pow()`	Các hàm toán thường dùng; để ý kiểu dữ liệu và tràn số.
`Serial.begin(baud)`	Khởi tạo tốc độ serial; tốc độ phía giám sát phải khớp.
`Serial.print/println()`	Gửi chuỗi/số lên terminal để theo dõi/debug.

6.4.4 Ví dụ: đọc analog và gửi Serial

Listing 6.2: Đọc ADC của Arduino Uno ATmega328P

```
const int ADC_PIN = A0;
void setup() {
  Serial.begin(9600);
}
void loop() {
  int raw = analogRead(ADC_PIN); // Uno: nominal 0..1023
  float voltage = raw * (5.0f / 1023.0f);
  Serial.print("ADC="); Serial.print(raw);
  Serial.print(" V="); Serial.println(voltage, 3);
  delay(250);
}
```

Công thức trên *giả định tham chiếu ADC đúng 5.0 V* và nguồn nằm trong mức an toàn, chỉ dùng để minh họa. Cần đo/tham chiếu Vref và xử lý sai số nếu đòi hỏi phép đo chính xác.

6.4.5 Serial UART và COM

Serial port truyền dữ liệu nối tiếp. Trong máy tính, “COM” có thể là cổng vật lý hoặc cổng ảo sinh bởi USB-to-serial, Bluetooth serial hoặc driver khác. Slide ghi chân 0 của Uno là RX, chân 1 là TX. UART TTL là mức logic của chip, khác *điện áp chuẩn RS-232*; không được nối trực tiếp phần cứng RS-232 truyền thống vào GPIO TTL nếu không có transceiver thích hợp. Mức baud, định dạng khung, GND và dây TX/RX phải được kiểm tra.

Theo Chapter 2: IoT Hardware, slide PDF 52–53.

6.5 ESP8266 và các module phổ biến

ESP8266EX (*Wi-Fi SoC*) của Espressif là vi mạch giá thấp tích hợp Wi-Fi và stack TCP/IP, có thể chạy ứng dụng độc lập hoặc đóng vai trò giao tiếp cho vi điều khiển khác. Slide đề cập

TCP, UDP, HTTP, mDNS, DNS, OTA, Flash/SD, SPI, I^2C và hệ Arduino core; việc hỗ trợ từng tính năng phụ thuộc firmware, core, thư viện và chân được đưa ra module. ESP8266 chủ yếu hoạt động ở mức logic 3.3 V: không cấp trực tiếp tín hiệu GPIO 5 V nếu không xác minh và chuyển mức.

Theo Chapter 2: IoT Hardware, slide PDF 54–61.

6.5.1 ESP-01, ESP-07 và NodeMCU

Bảng 6.4: Những dạng phần cứng ESP8266 xuất hiện trong slide

Module / bo	Điểm cần nhớ
ESP-01	Module nhỏ (slide nêu khoảng 24.75×14.5 mm), anten PCB, chân hạn chế, nguồn 3.3 V; firmware AT phổ biến cho điều khiển qua UART.
ESP-07	Module 3.3 V; slide nêu anten gốm và đầu nối U.FL, Wi-Fi 2.4 GHz b/g/n, TCP/UDP; chú ý chân pitch nhỏ.
NodeMCU	Bo phát triển dùng module ESP8266, có USB-UART, bộ ổn áp và đưa chân ra hàng header; sơ đồ nhãn D0/D1... không đồng nhất số GPIO silicon.

6.5.2 Firmware, SDK và quy trình IDE

Slide đề cập Espressif SDK NonOS, RTOS SDK, ESP-Open-SDK và các hệ Arduino/ESP Easy/ESPHome. Đây là các lựa chọn có lịch sử phát triển riêng, không phải cùng một thư viện. Đối với ví dụ lịch sử trên slide, cài ESP8266 Arduino core bằng cách thêm Boards Manager URL rồi cài package ESP8266 và chọn Generic ESP8266 Module; *URL và phiên bản package có thể thay đổi*, phải xem tài liệu chính thức hiện hành.

Theo Chapter 2: IoT Hardware, slide PDF 58–61.

6.6 ESP32: một họ SoC, không phải một chip duy nhất

ESP32 được Espressif phát triển với nhiều bộ xử lý, ngoại vi và khả năng vô tuyến tùy *dòng chip*. Slide mô tả ESP32 đời đầu có hai lõi, khoảng 160–240 MHz, hỗ trợ Wi-Fi/Bluetooth, nhiều GPIO và công năng kết nối cao hơn ESP8266, chi phí cao hơn. Tuy nhiên, có ESP32-S/C/H và các biến thể *không có cùng số lõi, Bluetooth, Wi-Fi, chân ADC hoặc giao diện USB*; không được áp dụng nguyên bảng thông số của ESP32 đời đầu cho toàn bộ họ ESP32.

Theo Chapter 2: IoT Hardware, slide PDF 62–65.

6.6.1 Lợi thế và giới hạn thực dụng

- **Tích hợp:** MCU và phần giao tiếp vô tuyến trên một chip/module giúp tiết kiệm diện tích và thời gian phát triển.
- **Ngoại vi:** GPIO, SPI, UART, I^2C, PWM, ADC, timer, interrupt; số lượng và ma trận ánh xạ chân phụ thuộc mã chip.
- **Nguồn:** phát Wi-Fi/BLE có dòng đỉnh cao; cần tụ decoupling và bộ ổn áp có dự trữ dòng. Giảm công suất bằng sleep hoặc giảm tần suất truyền khi tác vụ cho phép.
- **Phần mềm:** Arduino core thuận tiện học tập; ESP-IDF thích hợp khai thác phần mềm hệ thống, RTOS và ngoại vi chuyên sâu.

- **Tương thích:** cần kiểm tra chân bootstrap, chân dành riêng Flash, chân input-only (với một số chip), miền ADC, mức logic, loại giao thức Bluetooth cụ thể.

Ghi nhớ

Nhận định trên slide "ESP32 ra đời do ESP8266 thiếu bảo mật": nội dung bài giảng có đề cập như vậy, nhưng không nên xem đây là lời giải thích lịch sử đầy đủ. ESP32 còn mở rộng năng lực xử lý, ngoại vi và tính năng nền tảng. Các cơ chế bảo mật thực tế phụ thuộc dòng chip, boot configuration, firmware và cách triển khai sản phẩm.

6.6.2 So sánh các lựa chọn MCU căn bản

Bảng 6.5: So sánh định tính khi chọn nền tảng; không thay datasheet

Đặc điểm	ATmega328P/Uno	ESP8266	ESP32 (họ chip)
Kết nối mặc định	Không có IP/Wi-Fi trên MCU	Wi-Fi 2.4 GHz	Tùy dòng: Wi-Fi và/hoặc Bluetooth; không đồng nhất
Hệ logic	Bo Uno cổ điển 5 V	Chủ yếu 3.3 V	Chủ yếu 3.3 V
Năng lực xử lý	8-bit, RAM rất nhỏ	MCU 32-bit	Tùy dòng, nhiều ngoại vi
Lập trình	Arduino IDE, AVR-GCC	Arduino core/SDK	Arduino core/ESP-IDF
Phù hợp	Điều khiển cơ bản, học GPIO	Nút IoT Wi-Fi giá thấp	Nút IoT đa cảm biến, kết nối, Edge nhỏ

6.7 Intel Edison: ví dụ nền tảng lai có tính lịch sử

Theo slide, module Intel Edison có kích thước xấp xỉ $35.5 \times 25 \times 3.9$ mm, SoC Atom "Tangier" sản xuất tiến trình 22 nm, hai lõi Silvermont 500 MHz, lõi Quark 100 MHz, RAM 1 GB, eMMC 4 GB, hỗ trợ Wi-Fi, Bluetooth 4, USB và đầu nối 70 chân. Các bo mở rộng được giới thiệu gồm *breakout board* mức I/O 1.8 V với USB OTG, cổng cấp nguồn, sạc và cầu UART; *Arduino expansion board* đưa thêm chân kiểu Arduino (slide liệt kê 20 digital, 6 analog, UART, I^2C, SPI/ICSP, microSD).

Theo Chapter 2: IoT Hardware, slide PDF 66–68.

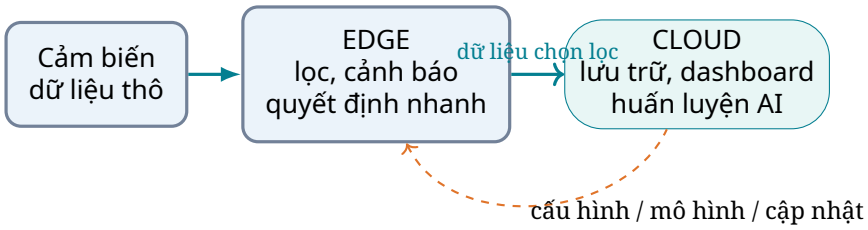
Ghi nhớ

Góc nhìn lịch sử: Intel Edison là nền tảng đã ngừng sản xuất/hỗ trợ. Chương này giữ để đọc hiểu slide và so sánh mô hình SoC + breakout + bo mở rộng, *không khuyến nghị* chọn làm nền tảng cho thiết kế sản phẩm mới. I/O 1.8 V cần đặc biệt chú ý khi nối module ngoài.

6.8 Edge Computing trong kiến trúc IoT

Điện toán biên (*edge computing*) chuyển một phần thu nhận, lọc, phân tích hoặc quyết định đến thiết bị gần nơi sinh dữ liệu hơn thay vì gửi toàn bộ dữ liệu thô lên đám mây. Bài giảng minh họa các tình huống *nhạy trễ*, *nhạy riêng tư*, bị hạn chế băng thông hoặc kết nối chập chờn; các thiết bị edge tiêu biểu có thể là MCU đủ mạnh, gateway Linux hoặc SBC.

Theo Chapter 2: IoT Hardware, slide PDF 69–72.



Hình 6.4: Sự phân công công việc điển hình giữa Edge và Cloud.

6.8.1 So sánh các vị trí xử lý

Bảng 6.6: Edge và Cloud: tiêu chí thiết kế

Tiêu chí	Xử lý Edge	Xử lý Cloud
Độ trễ	Có thể thấp, không nhất thiết phụ thuộc Internet	Có thêm trễ mạng và dịch vụ
Băng thông	Có thể lọc/gộp dữ liệu trước khi gửi	Có khả năng nhận tập dữ liệu lớn
Kết nối kém	Có thể tiếp tục một số chức năng	Khó dùng khi mất Internet
Tài nguyên	Giới hạn bởi công suất tính toán, pin, RAM	Co giãn dễ hơn, nhưng có phí/độ trễ
Riêng tư	Giảm dữ liệu thô rời thiết bị nếu thiết kế đúng	Cần chính sách truy cập và lưu trữ phù hợp

6.9 Raspberry Pi: máy tính đơn bo cho gateway và Edge

Raspberry Pi là họ máy tính đơn bo kích thước nhỏ, xuất hiện trong bài giảng như lựa chọn chạy hệ điều hành, kết nối mạng, làm gateway, hiển thị và phân tích dữ liệu. Khác với Uno, Raspberry Pi có thể chạy Linux và nhiều chương trình đồng thời, song khởi động chậm hơn, cần nguồn ổn định và bảo vệ lưu trữ. Slide đưa ví dụ Raspberry Pi 4 và những thành phần của kit: bo Pi, thẻ microSD, nguồn, bàn phím/chuột USB, màn hình HDMI, cáp HDMI, vỏ bảo vệ.

Theo Chapter 2: IoT Hardware, slide PDF 72–77.

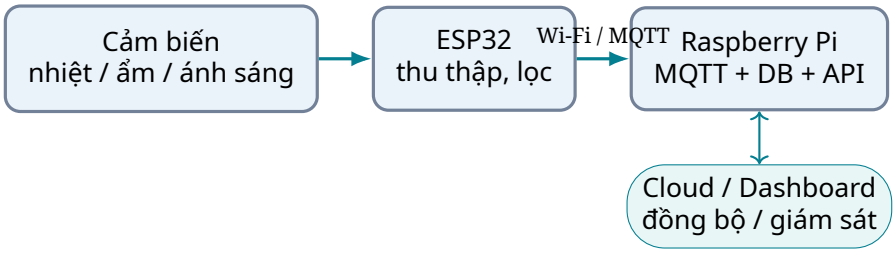
6.9.1 Các thành phần và GPIO

Raspberry Pi 4 minh họa trong slide có các khu vực: SoC, RAM, cổng USB, Ethernet, HDMI dạng micro, USB-C cấp nguồn, đầu camera/display và hàng chân mở rộng 40-pin. **GPIO của Raspberry Pi hoạt động theo logic 3.3 V**, không được đưa trực tiếp điện áp 5 V vào GPIO nếu không có mạch giao tiếp phù hợp. Số chân header vật lý, số GPIO và chức năng alternate không đồng nhất.

Cần phân biệt: USB-C trên Raspberry Pi 4 chủ yếu dùng cấp nguồn; điều đó không đồng nghĩa mọi Pi có thể lập trình/nạp trực tiếp như ESP32 qua cùng cổng. Với Pi chạy Linux, lập trình thường thông qua SSH, terminal, editor hoặc copy file lên hệ thống; môi trường và cách boot khác MCU.

6.9.2 Dựng gateway cơ bản với Raspberry Pi

Một kiến trúc ví dụ: cảm biến nối với ESP32, ESP32 truyền dữ liệu qua Wi-Fi/MQTT đến broker trên Raspberry Pi, Pi lưu cơ sở dữ liệu/đẩy lên dashboard/cloud. Nếu mạng Internet bên ngoài mất, dữ liệu có thể vẫn được lưu ở Pi trong LAN; khi có mạng lại, tiến hành đồng bộ theo chính sách ứng dụng.



Hình 6.5: Ví dụ nguyên lý mạng IoT tại nhà dùng ESP32 và Raspberry Pi.

6.9.3 Arduino và Raspberry Pi: không có nền tảng nào tốt nhất cho mọi việc

Bảng 6.7: Bảng kết luận đối chiếu nội dung cuối slide

Thuộc tính	Arduino Uno (ATmega328P)	Raspberry Pi 4 (SBC)
Bộ xử lý	MCU 8-bit điều khiển đơn giản	SoC mạnh chạy Linux
Hệ điều hành	Sketch chạy trên MCU, không có Linux	Hệ điều hành và đa tiến trình
Thời gian khởi động	Thường nhanh và đơn giản	Cần boot OS từ thẻ/lưu trữ
Tính xác định	Thuận lợi cho I/O định thời đơn giản	Linux thông thường không bảo đảm hard real-time
Mạng	Cần shield/module nếu muốn Wi-Fi	Có giao tiếp mạng tích hợp ở một số model
Ứng dụng	Cảm biến, đóng ngắt, điều khiển thời gian đơn giản	Gateway, database, dashboard, xử lý hình ảnh
Chi phí, điện năng	Thường thấp hơn ở lớp nút đơn giản	Thường cao hơn về điện năng/tài nguyên

Theo Chapter 2: IoT Hardware, slide PDF 78.

6.10 Thiết kế nút IoT: bài toán tổng hợp nhiều nền tảng

6.10.1 Bảng quyết định chọn phần cứng

Bảng 6.8: Checklist thiết kế từ yêu cầu tới linh kiện

Câu hỏi	Quyết định kỹ thuật tương ứng
Dữ liệu cần đo là gì?	Cảm biến, số trục, khoảng đo, accuracy, giao tiếp và tốc độ lấy mẫu
Tác động tới môi trường?	LED/servo/motor, driver, điện áp, dòng đỉnh và mạch bảo vệ

Câu hỏi	Quyết định kỹ thuật tương ứng
Làm việc khi mất mạng?	Đặt xử lý thiết yếu trên MCU/Edge; đệm dữ liệu và xác thực gửi lại
Tốc độ/khối lượng xử lý?	MCU 8-bit, ESP32, SBC Linux, bộ nhớ cần thiết
Cần Wi-Fi/BLE/RF?	SoC tích hợp hay module ngoài, băng thông, vùng phủ, anten
Nguồn pin hay nguồn cố định?	Dòng active/sleep/peak, nguồn ổn áp và sạc, thời lượng mục tiêu
Dung lượng dữ liệu?	RAM, Flash, EEPROM, microSD, độ bền ghi và khả năng phục hồi
Bảo mật?	Danh tính thiết bị, xác thực broker/API, TLS khả thi, firmware OTA an toàn
Độ ổn định vận hành?	Watchdog, reset, brownout, lọc nhiễu, xác minh wiring và logging
Bảo trì về sau?	Datasheet, sơ đồ chân, BOM, README, phiên bản firmware, test plan

6.10.2 Công thức sơ bộ về lưu trữ và truyền thông

Nếu một mẫu có B byte và tốc độ f_s mẫu/giây, trong thời gian T giây thì dữ liệu thuần tối thiểu:

$$\boxed{M = B\, f_s\, T} \quad \text{byte}, \tag{6.1}$$

chưa tính timestamp, JSON/CSV, header mạng, kiểm tra lỗi và hệ số dự phòng. Ví dụ mỗi mẫu 24 byte, 10 Hz trong 1 giờ: $24 \cdot 10 \cdot 3600 = 864{,}000$ byte (khoảng 0.864 MB hệ thập phân). Khi ghi microSD/Flash phải tính cả kích thước block và tuổi thọ ghi.

6.10.3 Ví dụ hệ IoT đầy đủ: giám sát phòng học

1. **Nhu cầu:** giám sát nhiệt độ/độ ẩm, độ sáng và thông báo nếu vượt ngưỡng; đồng thời quản lý quạt thông gió.
2. **Cảm biến:** chọn module nhiệt–ẩm số và cảm biến ánh sáng, đánh giá dải đo, sai số và điều kiện lắp.
3. **Bộ xử lý:** chọn ESP32 nếu cần Wi-Fi trực tiếp; chọn Uno kèm module Wi-Fi nếu cần học các khối tách rời; chọn Raspberry Pi làm gateway/dashboard.
4. **Actuator:** relay/MOSFET phù hợp điện áp, dòng tải và chuẩn an toàn; thiết lập trạng thái fail-safe nếu MCU lỗi.
5. **Phần mềm:** ước lượng chu kỳ lấy mẫu, chuẩn hóa dữ liệu, timestamp, MQTT hoặc HTTP, cấu hình ngưỡng và xử lý mất kết nối.
6. **Kiểm thử:** chạy đo trong điều kiện bình thường, biên cao/thấp, rút mạng, cấp nguồn lại, thử dòng đỉnh của tải và log lỗi.

6.11 Câu hỏi tự luyện và đáp án ngắn

1. **MCU khác SBC ở đâu?** – MCU tích hợp CPU, RAM, Flash và ngoại vi nhúng; SBC là hệ máy tính hoàn chỉnh thường chạy OS và có nhiều tài nguyên hơn.

2. **ATmega328P có mấy loại bộ nhớ chính theo slide?** – Flash 32 KB, SRAM 2 KB, EEPROM 1 KB.
3. **Arduino Uno cổ điển có bao nhiêu ngõ digital và analog?** – 14 digital và 6 analog được đưa ra chân trên bo, không đồng nghĩa chip chỉ có từng đó chức năng.
4. **Vai trò `setup` và `loop`?** – Cấu hình một lần; vòng xử lý lặp liên tục.
5. **Serial port chân nào trên Uno?** – D0/RX và D1/TX; GND và mức điện áp phải tương thích.
6. **ESP-01 khác NodeMCU?** – ESP-01 là module nhỏ với ít chân; NodeMCU là bo phát triển tích hợp USB-UART/nguồn, tiện lập trình.
7. **ESP32 có luôn dual-core và Bluetooth Classic?** – Không; đặc tính tùy series/model.
8. **Tại sao Intel Edison xuất hiện trong bài?** – Ví dụ lịch sử về module SoC + breakout/Arduino board; không dùng làm lựa chọn hiện đại mặc định.
9. **Khi nào cần Edge?** – Khi ưu tiên trễ thấp, giảm băng thông, bảo vệ riêng tư và duy trì tác vụ tại chỗ khi mất mạng.
10. **Raspberry Pi khác Arduino về môi trường chạy?** – Pi thường chạy Linux; Uno chạy sketch nhúng, không có hệ điều hành đa nhiệm tổng quát như Linux.

6.12 Bảng thuật ngữ ôn thi

Thuật ngữ	Diễn giải vắn tắt
MCU	Microcontroller Unit – vi điều khiển đơn chip tích hợp CPU, bộ nhớ, ngoại vi.
SoC	System-on-Chip – hệ thống tích hợp trên chip, thường có CPU, ngoại vi và khối chuyên dụng.
SBC	Single-board Computer – máy tính trên một bo, thường chạy OS.
GPIO	General-Purpose Input/Output – chân nhập/xuất số đa dụng.
ADC	Analog-to-Digital Converter – bộ chuyển tương tự sang số.
PWM	Pulse-Width Modulation – điều chế độ rộng xung.
UART	Universal Asynchronous Receiver/Transmitter – nối tiếp bất đồng bộ.
SPI	Serial Peripheral Interface – giao tiếp nối tiếp đồng bộ, thường có CS.
I^2C	Bus SDA/SCL hai dây, nhiều thiết bị theo địa chỉ.
Bootloader	Phần mềm khởi động hỗ trợ nạp firmware qua giao diện quy định.
OTA	Over-the-Air – cập nhật firmware qua mạng vô tuyến.
Shield	Bo cắm mở rộng cho một nền tảng (ví dụ Arduino).
Breakout board	Bo đưa các chân/tính năng của module/chip ra chân kết nối dễ dùng.
Edge Computing	Xử lý ở gần nơi sinh dữ liệu thay vì chuyển hết về Cloud.
MQTT	Giao thức publish/subscribe nhẹ phổ biến trong IoT, sử dụng broker.

Ghi nhớ

Tóm tắt cần nhớ: Phân biệt *MCU – module – board – SBC*. ATmega328P/Arduino thích hợp nền tảng học điều khiển; ESP8266/ESP32 bổ sung kết nối không dây và năng lực xử lý; Intel Edison là tư liệu lịch sử; Raspberry Pi hợp lý cho gateway và ứng dụng Linux. Edge giúp giảm độ trễ và phụ thuộc Cloud. Luôn đối chiếu sơ đồ chân, điện áp logic và nguồn trước khi lắp mạch.

Tài liệu tham khảo và quy ước trích dẫn

[1] Nguyễn Đắc Cử, *Cơ sở và thiết kế hệ thống IoT – Chapter 1: An overview of IoT*, bài giảng 50 trang PDF, Trường Đại học Phenikaa. **Nguồn chính**; số trang nêu trong nội dung là số trang PDF.

[2] Qusay F. Hassan (chủ biên), *Internet of Things A to Z: Technologies and Applications*, IEEE Press/Wiley, 2018. Tham khảo Chương 1 (smart object, smart environment, ICT) và Chương 3 (kiến trúc, công nghệ nền).

[3] Adrian McEwen và Hakim Cassimally, *Designing the Internet of Things*, Wiley, 2014. Tham khảo Chương 1 (tư duy thiết bị kết nối) và Chương 2 (nguyên tắc thiết kế và quyền riêng tư).

[4] Alice James, Avishkar Seth và Subhas Chandra Mukhopadhyay, *IoT System Design: Project Based Approach*, Springer, bản in ghi 2022. Tham khảo Chương 1 (khối chức năng IoT) và Chương 3 (cân nhắc thiết kế nút IoT).

[5] Nguyễn Đắc Cử, *Cơ sở và thiết kế hệ thống IoT – Chapter 2: IoT Hardware*, bài giảng 79 trang PDF, Đại học Phenikaa. Nguồn chính cho chương 4–6.

Quy ước: “Theo bài giảng” là nội dung được diễn giải lại từ slide; “Kiến thức mở rộng” là giải thích bổ trợ do tài liệu này biên soạn dựa trên các sách tham khảo hoặc kiến thức kỹ thuật chung. Không dùng hình từ PDF gốc trong bản phân phối.